



PDF Download
3586183.3606719.pdf
15 January 2026
Total Citations: 47
Total Downloads: 1469

Latest updates: <https://dl.acm.org/doi/10.1145/3586183.3606719>

RESEARCH-ARTICLE

Spellburst: A Node-based Interface for Exploratory Creative Coding with Natural Language Prompts

TYLER ANGERT

MIROSLAV SUZARA, Stanford Graduate School of Education, Stanford, CA, United States

JENNY HAN, Stanford University, Stanford, CA, United States

CHRISTOPHER PONDOC, Stanford University, Stanford, CA, United States

HARIHARAN SUBRAMONYAM, Stanford Graduate School of Education, Stanford, CA, United States

Open Access Support provided by:

Stanford Graduate School of Education

Stanford University

Published: 29 October 2023

[Citation in BibTeX format](#)

UIST '23: The 36th Annual ACM Symposium on User Interface Software and Technology
October 29 - November 1, 2023
CA, San Francisco, USA

Conference Sponsors:

SIGCHI
SIGGRAPH

Spellburst: A Node-based Interface for Exploratory Creative Coding with Natural Language Prompts

Tyler Angert^{*}
tyler@replit.com
Replit
San Francisco, USA

Miroslav Ivan Suzara^{*}
msuzara@stanford.edu
Stanford University
Stanford, USA

Jenny Han^{*}
jennyhan@cs.stanford.edu
Stanford University
Stanford, USA

Christopher Lawrence Pondoc
clpondoc@stanford.edu
Stanford University
Stanford, USA

Hariharan Subramonyam
harihars@stanford.edu
Stanford University
Stanford, USA

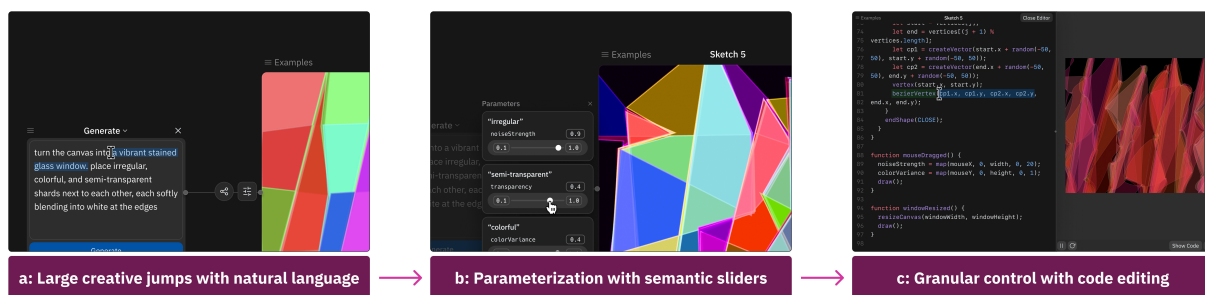


Figure 1: Exploratory Creative Coding using Spellburst: (a) the artist uses a natural language prompt to generate an initial sketch based on semantic intent, (b) uses the dynamic prompt-driven slides to rapidly refine the output, and (c) performs fine-grained adjustments through direct code-editing.

ABSTRACT

Creative coding tasks are often exploratory in nature. When producing digital artwork, artists usually begin with a high-level semantic construct such as a “stained glass filter” and programmatically implement it by varying code parameters such as shape, color, lines, and opacity to produce visually appealing results. Based on interviews with artists, it can be effortful to translate semantic constructs to program syntax, and current programming tools don’t lend well to rapid creative exploration. To address these challenges, we introduce Spellburst, a large language model (LLM) powered creative-coding environment. Spellburst provides (1) a node-based interface that allows artists to create generative art and explore variations through branching and merging operations, (2) expressive prompt-based interactions to engage in semantic programming, and (3) dynamic prompt-driven interfaces and direct code editing to seamlessly switch between semantic and syntactic exploration. Our evaluation with artists demonstrates Spellburst’s potential to

enhance creative coding practices and inform the design of computational creativity tools that bridge semantic and syntactic spaces.

CCS CONCEPTS

• **Human-centered computing** → Natural language interfaces; Graphical user interfaces; • **Software and its engineering** → Software version control; • **Applied computing** → Fine arts.

KEYWORDS

large language models, exploratory programming, creative coding, generative art, prompt engineering

ACM Reference Format:

Tyler Angert^{*}, Miroslav Ivan Suzara^{*}, Jenny Han^{*}, Christopher Lawrence Pondoc, and Hariharan Subramonyam. 2023. Spellburst: A Node-based Interface for Exploratory Creative Coding with Natural Language Prompts. In *The 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*, October 29–November 01, 2023, San Francisco, CA, USA. ACM, New York, NY, USA, 22 pages. <https://doi.org/10.1145/3586183.3606719>

1 INTRODUCTION

Creative coding is an *expressive* and *exploratory* activity. When producing artwork, artists often begin with high-level constructs grounded in mimesis or metaphors and programmatically explore digital renditions of their ideas [21, 24, 40]. For instance, imagine

^{*} indicates equal contribution by authors.

creating art inspired by the reflective behavior of light. Starting with the idea of stained glass windows, the artist may write code to generate a glass effect on the digital canvas. They may then rapidly explore variations by adjusting the color and translucency of the glass, angle of light reflection, amount of refraction, etc. Finally, based on outcomes, they may take a larger creative jump and explore a different construct, such as the reflection of moonlight on water. Across these exploratory tasks, artists (1) repeatedly translate expressive semantic intents into lines of code, (2) keep track of different creative variations, and (3) compare, combine, and extend emergent outputs to further exploration. Unfortunately, existing programming environments don't lend themselves well to this form of creative exploratory process [6]. While the needs of artists engaged in creative coding tasks overlap with traditional programming, they require specific affordances for rapid iteration and semantic exploration.

First, creative coding is experimentation-heavy. During exploration, artists would benefit from systems that help them quickly and intentionally keep track of intermediate visual outputs and corresponding specifications. Current versioning strategies are largely disconnected from programming environments and demand a separate workflow to commit and retrieve program states. This introduces friction in artists' creative processes to document outputs for later consideration. Artists may have to repeatedly capture screenshots of their evolving art and manually associate them with corresponding code either through versioned code files or commented code snippets. Second, given the emphasis on *visual* outputs as feedback and inspiration for subsequent exploration, artists would benefit from ways to visualize and compare generated outputs. Current programming tool layouts primarily consist of a side-by-side text editor and output view, making it challenging to visually compare results across multiple "runs" of the program. While recent tools address issues of comparing different outputs [37, 74, 83], they are still primarily disconnected from generative coding tasks (e.g., combining line attributes from one output with color from a different exploratory path) [7].

Third, as mentioned in the light reflection example, larger creative jumps are often grounded in semantic constructs. When such jumps occur during creative thinking, artists spend a significant amount of time programmatically authoring initial representations of the new idea. Consequently, they may take fewer creative shifts and fixate on incremental variations of their initial ideas [39, 53]. An advantage of the recent generative AI tools based on large language models (LLMs) is that they allow artists to quickly explore divergent ideas through expressive natural language prompts (e.g., Adobe Firefly [2]). However, finer editing using natural language prompts can be tedious (e.g., "increase the line thickness by 5 pixels"). Artists benefit from executing larger creative shifts using semantic prompts, while program syntax (or dedicated interface controls) may be better suited for fine-grained exploration and control. Thus, the motivating question for our work is *how can we incorporate generative AI capabilities within current creative programming workflows to support exploratory generative art-making?*

By conducting formative interviews with ten expert creative coders, we identified ways in which artists navigated semantic and syntactic spaces, distinct tasks in their creative exploratory workflows, and pain points due to limitations of existing tools. Based on

insights from the study, we developed *Spellburst*, a visual interface for exploratory programming. The primary view in *Spellburst* is a *node-based* layout in which each node represents a creative output along with backing code. Artists can use branching and merging interactions to clone, extend, and combine different variations. As shown in Figure 1, to execute larger semantic jumps, *Spellburst* supports a natural language *prompting interface* to generate code, and artists can further refine the generated code either through dynamic prompt-driven interface controls or directly edit the code in the code editor. We conducted an online survey with 52 crowd-sourced participants to understand the range of natural language prompts for specific creative tasks. Based on insights from the survey, we implemented *Spellburst* to support seamless switching between prompt-based exploration and program editing, and the visual layout helps them keep track of the creative exploration process. We evaluated *Spellburst* with 10 expert artists across controlled and open-ended creative tasks.

We contribute (1) *Spellburst*, a node-based visual programming interface for exploratory programming tasks, (2) expressive prompting support for creative experimentation, and (3) interface affordances for semantic-syntactic integration to flexibly execute larger creative shifts and fine-grained code-editing.

2 RELATED WORK

Creative coding is a branch of computer programming where the purpose is to create *artistic* and *expressive* output rather than functional output [58, 68]. *Generative art* is a specific approach to creative coding in which artists "program computers to undertake creative instructions" [16, 75, 79]. Mathematical and computational algorithms are central to this approach. In producing art, artists engage in the exploration and experimentation of code to produce variations of algorithms and outputs [6, 35, 36]. During this process, they encounter phenomena such as emergence, randomness, and interaction to produce creative results [16, 59, 60, 79].

Today, artists may use programming tools such as Processing [62], p5.js [48], OpenFrameworks [44], and TouchDesigner [15] to create their artwork. Within these tools, exploration by programming takes place through *edit-run cycles* in close proximity [35, 36, 50]. However, this can also vary by scale (i.e., changing a specific variable versus file-level changes) and duration (i.e., pertaining to one particular element versus a broader computational model that is being explored) [6]. This contrasts with traditional conceptualizations of programming, where specifications are typically mapped out in advance and for functional purposes. In *Spellburst*, our goal is to support open-ended creative experimentation.

We narrow our focus to three specific needs for creativity support tools (CST) within the realm of exploratory creative coding: (1) version control systems for *history-keeping* [71], (2) higher *closeness of mapping* between semantic concepts and syntactic representations in creativity support tools [6, 23], and (3) AI augmentation for creative exploration [19, 71].

2.1 Tracking Exploration History

A user's personal code history is an important site for exploration and for management of alternatives, especially in creative settings [71, 72]. Existing history-keeping or version control systems

such as Git offer features for documenting previous alternatives but are costly to use in terms of time and effort [6, 12, 36]. Creative coders might prefer informal or manual versioning methods such as taking screenshots or screen captures of each run and copy-pasting code [36, 43, 72]. Second, current version control systems are not explicitly designed for rapid iteration and remixing on prior states. Creative work is often non-linear; artists jump back to previous work or explore multiple ideas in parallel [43]. Using tools such as Git to create new branches, revert to previous states, or merge branches creates friction that disrupts an artist’s flow or process.

Previous qualitative studies with artists emphasize the importance of designing and co-designing with artists’ mental models for code and code versions in mind [43, 61, 72]. Like artists who work with physical media, generative artists also engage deeply with the materiality of their work [61]; they pay special attention to the properties, expressiveness, and craftsmanship of their software [56, 61]. They may view their versions as panels to “pin up” for review [12], or they may think of their versions as a “palette of materials” with which to mix, experiment, and use [72].

Existing work on novel version control systems for coding has aimed to support rapid iteration and exploration by allowing parallel source editing and execution [26], providing a sandbox environment to quickly test, save or discard “microversions” of code [36, 50], and storing each commit as a node in a graph alongside annotations, screenshots, or other key semantic information [12, 29, 61]. Spellburst builds on this work in a new direction by using AI to support iteration and manage code versions.

2.2 Support for Expressive Programming

Creative coders work across a semantic-syntactic divide, as described by Reas [60]. Artists often envision and explicate their work through visual metaphors [17] or rich verbal descriptions [76]. When using programming languages to create art, artists must then map expressive intents in the semantic space (i.e., “undulating waves of color,” a doodle of a design) to low-level program syntax (i.e., p5.js functions such as `lerp()` or `sin()`) [60, 79]. However, effectively mapping semantic concepts to low-level code can be difficult for creative coders of all levels [32, 77]. Prior research in visual programming languages (VPLs) [52] have looked at reducing friction by presenting some components of the code via two-dimensional visuals. For example, block-based programming languages such as Scratch [46] or node-based programming languages such as TouchDesigner, Stamper [12], and natto.dev [67] help users abstract important aspects such as control flow, variables, and data into visual representations. By leveraging advances in computer vision, systems such as Sketch2Code [33] directly map hand-drawn sketches to HTML code.

Recent advancements in generative AI and text-to-code models such as Codex and Copilot have allowed users to write code using natural language prompts and autocomplete features within integrated development environments (IDEs) such as VSCode [1] and Replit [63]. LLMs have accelerated the progress in bridging semantic and syntactic space by autonomously mapping natural language prompts to code output. Anecdotally, ChatGPT has already been integrated into common creative tools such as Unity and Adobe

Suite. However, publically available LLMs are not necessarily optimized for a creative coding context. Mapping parameters within a high-dimensional design space to human language and feedback requires domain-specific frameworks [69].

While VPLs and natural language tools provide more accessible representations of code, direct manipulation interfaces provide more accessible *interactions* with code. One example of a system with a physical direct manipulation interface is Dynamic Brushes, which maps input data from a physical stylus to digital properties such as position, aesthetic style, and geometric transformations [31]. Other systems involve digital interfaces for manipulation, the simplest of which involve sliders, toggles, and tuners within a block of code to allow users to tweak parameter values [26, 77]. Drawing on this body of work, Spellburst incorporates visual node-based programming, natural language prompting with auto-complete suggestions, and direct manipulation interfaces in an attempt to bridge the semantic-syntactic divide for generative artists.

2.3 AI Augmentation for Creative Tasks

In creative thinking, artists draw inspiration from others’ work [71]. Research shows that novel ideas may arise from the “genetic recombination” of many people’s work [38]. In Spellburst, we are interested in how AI, particularly LLMs, can augment human creative capabilities and inspire new forms of artistic expression as a collaborator. Text-to-Image (TTI) models such as Midjourney, Stable Diffusion [64], and DALL-E [55] have proliferated, but our mental models for interacting with prompt- and chat-interfaces are nascent. For example, given the stochastic nature of the models, it may take several generations of the same prompt to provide a suitable output [45]. The black-boxed nature of the TTI models does not always allow for low-level control, which may be frustrating to some artists [41]. One current way artists ensure somewhat reproducible and desired results is by creating a prompt template [13]; for example, a simple template might be “SUBJECT in the style of STYLE” [45]. Overall, research has shown that prompt-based LLMs are useful for fast iteration and quick combination of ideas [41], and early work points to potential for novel interfaces for multi-modal input, iterative inputs, and gesture-based input [41]. Spellburst focuses on iterative and non-linear prompts in particular.

Auto-complete interfaces for code, such as Copilot and Ghostwriter, are becoming more powerful [4, 51]. Such AI assistants help programmers in two ways: acceleration (i.e., when the user already knows what to do) and exploration (i.e., when a user does not know what to do next) [4]. However, it is important to consider the additional cognitive load on users to process auto-complete recommendations from the AI. When such tools were in use, the most time-consuming task for developers became “verifying/thinking suggestions” from the AI, which highlights the importance of a cognitively informed workflow.

3 FORMATIVE INTERVIEWS WITH EXPERTS

To better understand the challenges faced by creative coders and inform the design of Spellburst, we conducted need-finding interviews with 10 expert creative coders. The study interrogated how artists currently set up their exploratory creative workflow, their

approach for navigating between semantic and syntactic spaces during exploration, and how they manage iterations.

3.1 Method

We interviewed 10 experts in creative coding who identified as generative artists ($n = 4$), software engineers ($n = 2$), visual designers ($n = 2$), game developers ($n = 1$), and creative coding educators ($n = 1$). Detailed demographic information can be found in the appendix (section A.6). Participants were recruited through word of mouth, social media posts, and snowball sampling. 7 of our participants were male, and 3 were female. All participants had 3+ years of programming experience so as not to conflate exploratory programming practices and novice programming practices [36]. 90% of our participants had 5+ years of programming experience. Each semi-structured interview lasted between 45 minutes to an hour, and participants were compensated with \$25 for their time.

In the first part of the interview, we asked questions related to participants' creative processes. Example questions included "How do you explore new creative options in your work?", "How do you iterate on your current work", "Can you tell us a time when you went back to a previous version of your code?" In addition, we asked participants about their approach to versioning and history tracking and their experience co-creating art with AI, if any. Next, as a contextual inquiry task, we asked participants to show us a recent creative project and walk us through their virtual workspace, including code files, IDE, and output viewing interfaces. 8 out of 10 participants chose to share a creative coding project; 1 spoke about an art project in Figma, and 1 spoke about an animation project in Adobe AfterEffects.

In the final part of the interview, we presented participants with a 20-minute open-ended creative coding task to see how they engaged in exploratory programming in real-time; the task required each participant to create a digital brush using mouse interaction based on simple starter code in p5.js. During this task, participants shared their screen via Zoom and used a think-aloud protocol to narrate their thoughts as they iterated on the starter code using Replit, an online IDE. Video recordings of the sessions were then transcribed. Two members of the research team watched all the videos and included relevant screenshots at appropriate points in the transcript documents. The same two members then read through all of the interview transcripts and conducted inductive coding [73] on ATLAS.ti [3] and identified key insights related to iteration, creativity, and exploratory programming. Through multiple rounds of discussions, we clustered the insights into high-level themes via affinity clustering and thematic analysis [9].

3.2 Findings and Design Considerations

3.2.1 Parameter- and accident-driven exploratory programming. Many of the participants described their exploratory process as parameter-driven, in which their exploration is based on tweaking a set of variables, which U9 referred to as "magic constants." U4 organizes their code by placing all of their parameters at the top of the file: *"I don't really have a great standing process for tracking versions, and I kind of just make a mental note, or just write down parameters that lead to something that I like...I like to chunk and get all the things that I would be modifying in one place and get all*

the things that are fairly static out of the way." When asked how they knew that they were iterating on parameters in the right direction, many participants pointed to previous experience. They explained that experts typically drew from previously used parameters from prior work. One commonly used pattern that experts used in the open-ended task involved constraining circular movement using the parameter range of $-\pi/4$ to $\pi/4$ radians. Therefore, CSTs should allow **artists to maintain fine-grained control over the parameters they are manipulating (Design Consideration 1).**

Other times, iterations on parameters came about through a process of trial-and-error, which some participants termed "accident-driven development." U3 describes a time when she uncovered a new idea in a p5.js sketch: *"I messed up what I was connecting, and then I ended up with two objects connected in the middle...so I think the little accidents are the inspiration."* During such discovery, surfacing parameters while running and testing code was another point of conversation; this was sometimes done through console statements or small helper functions. U7 described writing "debug" functions which programmed hotkey buttons to show him the range of possible outputs while he was running a 3D game simulation, especially in situations where controlled randomness was required. U5 described the cycle of iterating on a particular parameter, running the code, and analyzing the result as a feedback loop: *"If I had like a little slider to make [this particular parameter] more or less like I was just tuning constantly...If there was a way to make that feedback loop quicker, that'd be perfect."* Thus, CSTs should **allow participants to read and manipulate parameter values (D2) easily and readily see the output to support rapid creative exploration (D3).**

3.2.2 Version control practices. Because creative exploration is often brought about by unintended changes, participants described mental friction between the exploratory process and the intentional process of committing and summarizing changes via version control systems like Git. In both parameter- and accident- driven development, tightly bound cycles of iteration, execution, and analysis were observed: *"So I do a lot of it in my head or like taking notes and stuff like that, and so I'll run it [and] see what happens"* (U7). Most participants chose not to use a version control system when creating personal projects despite having familiarity with Git. U10, one of the few who did use Git for creative work, described the friction using Git commits and branches in this way: *"It just required too much planning ahead when this process [creative exploration] is not about planning ahead; it's about seeing where you go."*

Instead, many participants described their informal versioning practices (similar to [36]). As a workaround, participants would duplicate files or code snippets and comment them out when not in use. Alternatively, three other participants described a process of tracking changes in working memory or by jotting notes down, saying some variation of "I just remember what I did previously": *"I do a lot of like...open up notepad and write down what was good [while running], and then, just remember that so it's definitely more of a conceptual than a version control thing"* (U7). Therefore CSTs should provide **versioning and tracking as an integrated experience with the rapid exploration process (D4).**

In addition to tracking code configurations that produced salient outputs, participants also used informal versioning practices to

keep track of the visual outputs of the code. Participants would track changes by running and storing the outputs via screenshots, and video captures (e.g., “versionFinal.jpg”, “versionFinalFinal.jpg”). While these were arguably lower fidelity than Git commits that included code [72], participants found this helpful to capture the semantic constructs that they were experimenting with at the time. As U3 described: “So the way I iterated was whenever I had something distinct enough that I wanted to save, I would just duplicate the p5 file and name it, then move on... So, from one generation to the other, I probably just saved it duplicated and then started working on top of that. It’s very jank version control.” However, the outputs were often detached from code or any other useful history-related metadata. In this way, CSTs should provide ways to effortlessly **track the emergent visual outputs during exploration along with the underlying code that produced the output (D5).**

3.2.3 Interest in AI Augmented Creative Exploration. The interviews were conducted in the summer of 2022. At that time, ChatGPT was not yet publicly available, but most artists had heard of and experimented with TTI models, code generation with Copilot, and other LLMs. Most participants were unsure how they felt about using AI in their personal projects. U3 stated that she “used to be very much against it” and would only consider using it after the bulk of the creative work was completed. U10 similarly felt AI would be good for “post-processing.” He thought about AI as a way to add “filters” on top of his work: “I don’t want to use [AI] for the core of my thing, but I could see using it for enhancing whatever I just did. So like if I created some pattern, I could [ask AI to] give it a great bloom effect or add a bunch of film grain.”

Participants did feel comfortable and excited about using AI to generate expressive prompts for inspiration, as it could help them explore new creative directions and overcome creative blocks. When asked if they could imagine iterating on creative code with a computer, they emphasized the importance of the human leading the creative process. U9 compared the utility of AI to a randomize feature that provides suggestions for new parameters for him to try. U10 suggested that the AI could be trained on his personal history: “Maybe it could learn from myself. All the other tools are learning from the world at large and existing art. I wonder if it could learn from the way that I do things. I wonder if it could be throwing up suggestions or provocations... side by side [while I code].”

In summary, AI-augmented CSTs should **offer different types of AI support for different parts of the creative process (beginning vs. iteration vs. end) and allow the artist to maintain creative control at all times (D6).**

4 USER EXPERIENCE

Spellburst is designed to facilitate rapid exploration and visualization of a large tree of possibilities while simultaneously preserving exploration history (D4 and D5). As shown in Figure 2, Spellburst’s user interface consists of an infinite zoomable canvas containing one or more *node-based* layouts. The node layouts follow a hierarchical tree structure, and each node is either a **sketch** or an **operator**. **Sketch** nodes represent p5.js [47] programs that are compiled and rendered to a canvas. The root node indicates a starting sketch for a creative path. **Operator** nodes are *links*

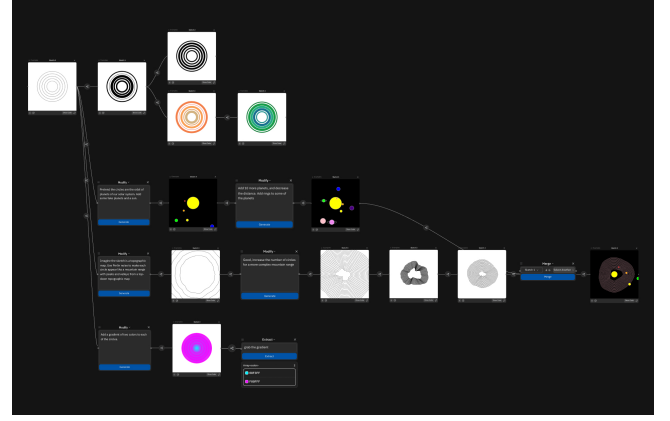


Figure 2: Overview of Spellburst’s User Interface

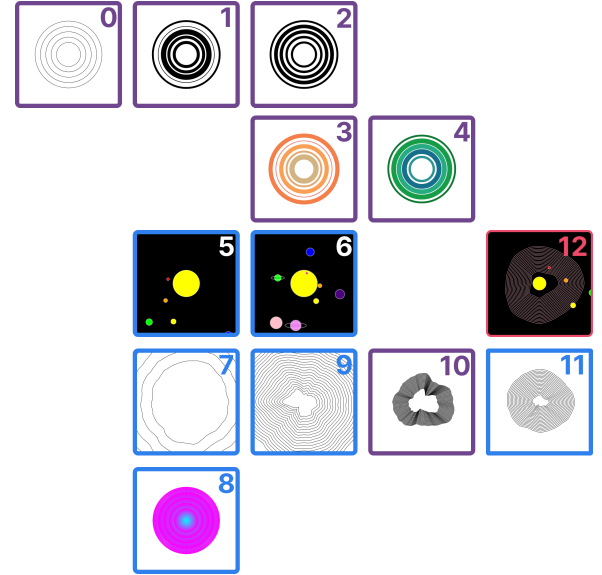


Figure 3: Sketches created by Ash during an exploratory creative coding session, color-coded by node type.

that indicate the following exploratory programming operations: modification, merging, duplication, diffing, and extraction. Some of these operator nodes allow natural language inputs; all of them output new sketches based on source sketches and prompts.

To demonstrate our system’s features and user experience, we describe how Ash, a creative coder, creates and iterates upon a generative art project using Spellburst.

4.1 Set-up

Ash has a conceptual idea about *concentric circles* to inspire her exploration. She opens Spellburst on her web browser, which presents a blank canvas with a floating add node ‘+’ button (Figure 4 a). Ash clicks on the button to create a root node, which is a **sketch**. Next, she clicks on the ‘show code’ button on the root node, which brings up the code editor to the left of the canvas (Figure 4 b). Here, she

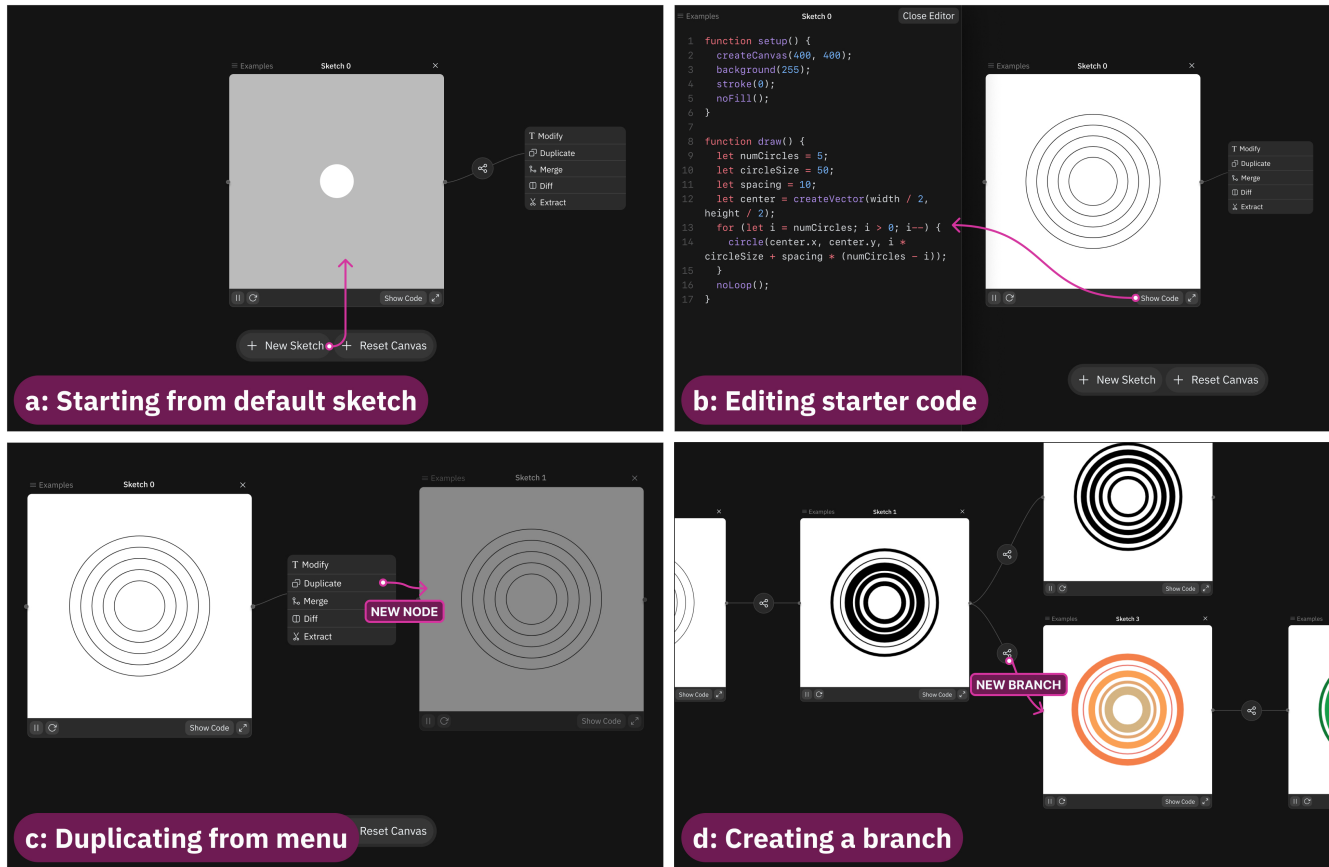


Figure 4: Setup and basic high-level controls of Spellburst

can author an initial sketch from scratch or copy code for concentric circles from sources on the web (such as the p5.js [47] gallery). Ash pastes the p5.js code snippet to render four concentric circles from the web, which displays the output on the root node (Figure 3 Sketch 0). Spellburst also has an ‘Examples’ tab with a set of starter code for basic shapes.

4.2 Focused Exploration

Starting with the initial root `sketch` node, Ash clicks on the button on the right edge of the node and selects the `duplicate` operator in the popup menu (Figure 4 c). This creates a copy of the root node and is visualized as a child `sketch` node to the root node. Compared to platforms like Fork It [81], Spellburst is designed to encourage iteration and “accident-driven development” rather than just executing nodes in parallel. Thus, all `sketch` nodes are connected to an operations menu, where selecting an option is required to continue adding nodes to the branch rather than being purely open-ended.

In the newly created copy, Ash modifies the sketch’s code in the code-editor to algorithmically randomize the line thickness. By repeatedly making copies of the previous node, Ash explores

different approaches to varying line thickness (Figure 3 Sketches 1 and 2). At this point, Ash decides to explore color variations.

However, she wishes to explore colors separately from line thickness. Selecting one of the previous sketches as the base, she initiates a `branch` operation by selecting from the button on the edge between the original sketch and its duplicate node (Figure 4 d). Note that unlike platforms like natto.dev [67], our edges aren’t freeform and arbitrary. They are generated from duplicating and iterating on existing sketches and prompts rather than being derived from code dependencies. In this new branch, Ash explores variations of color by directly editing the attributes and functions in the code editor, which produces Sketches 3 and 4 in Figure 3. As we will see later, Ash can also use natural language prompts to execute these changes, but based on our interviews, artists prefer greater control over familiar attributes (D1 and D6).

4.3 Exploring Large Creative Jumps

At this point, Ash imagines a new idea of circles representing the planetary orbits of our solar system. She decides to use Spellburst’s natural language prompt to explore this new direction. Clicking on the initial `sketch` node, Ash initiates a `modify` operator, which brings up a prompting interface along the link (Figure 5 a).

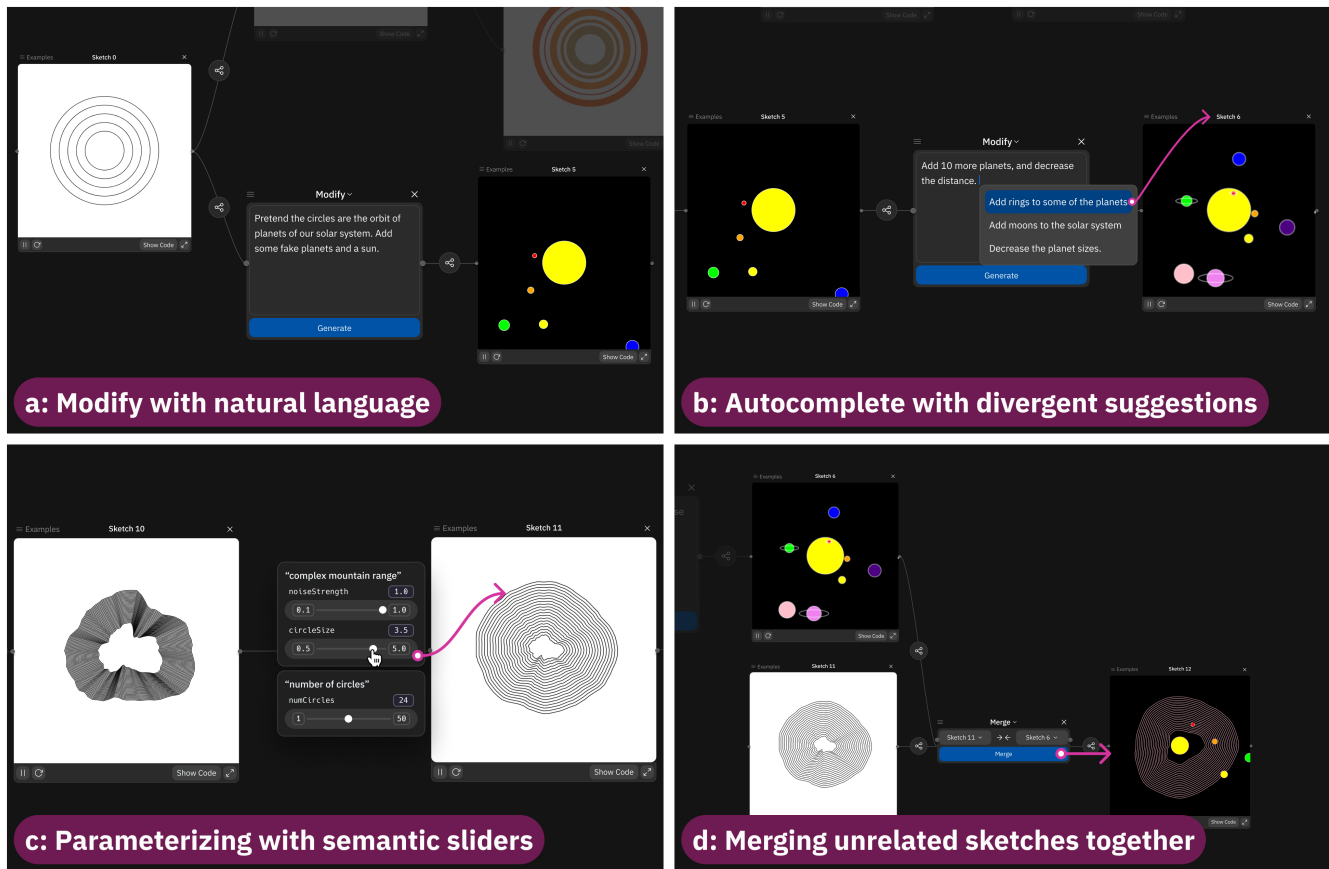


Figure 5: Modification, merging, and semantic sliders powered by LLMs

In the text box, Ash enters the prompt “Pretend the circles are the orbit of planets of our solar system. Add some fake planets and a sun.” Using the previous sketch and the prompt as inputs, Spellburst generates a new `sketch` node with an animated output showing several concentric circles and elliptical objects of different sizes animating along the orbits (Figure 3 Sketch 5). Such a large creative jump would have been time-consuming to manually program.

Ash inspects the generated output and finds it interesting, so she decides to continue with this new direction. She creates a new `modify` operator and begins to write a new prompt: “Add 10 more planets, and decrease the distance.” To help Ash with her exploration, Spellburst provides inspiration through an auto-complete drop-down (Figure 5 b), which suggests three additional ideas: “Add rings to some of the planets,” “Add moons to the solar system,” and “Decrease the planet sizes.” She hadn’t thought about adding rings, but she likes the idea and selects this auto-complete option resulting in a new `sketch` node (Figure 3 Sketch 6).

Later, Ash returns to the original sketch and explores two entirely different concepts: one related to Perlin noise and topographic maps (Figure 3 Sketches 7,9,10,11) and another related to color gradients (Figure 3 Sketch 8). These are depicted as two additional branches from the initial sketch node. Ash uses the prompts “Imagine the

sketch is a topographic map. Use Perlin noise to make each circle appear like a mountain range with peaks and valleys from a top-down topographic map” to produce Sketch 7 and “Good, increase the number of circles for a more complex mountain range” for Sketch 9.

4.3.1 Iterating on generated outputs through code editing. By looking at the output for the topographic maps, Ash is not quite satisfied with Sketch 9. In this instance, prompt-based refinement can be tedious. Therefore, Ash decides to iterate directly on the code on a more granular level. As before, she clicks on the ‘show code’ button to open up a code editor view of Sketch 9. Spellburst automatically provides code comments for the prompt-generated code. She clicks on the ‘maximize’ button to focus the interface and zoom in on just that node (similar to Figure 1 c). She manually updates the values for the spacing parameter to create a denser arrangement, which instantly updates the output view. Once Ash is satisfied with this version, she zooms back out to the canvas, and adds a `duplicate` operator to make a copy of Sketch 9 so she can manually edit the code; she creates Sketch 10 this way.

4.3.2 Iterating on prompts using Semantic and Global Variable Sliders. To provide fine control over the prompt phases or propositions, Spellburst has auto-generated various sliders for global variables

present in the code: `circleSize`, `numCircles`, and `noiseStrength`. Further, she notices that Spellburst has automatically mapped relevant words and phrases from the original modification prompt to these variables, allowing her to adjust the relative influence of words in the prompt and see changes to the sketch in real-time (Figure 5 c). Ash plays around with both of the sliders, producing Sketch 11. These dynamic prompt-driven sliders, in addition to our non-deterministic autocomplete, allow Spellburst branches to easily enable quick exploration of extremely divergent creative paths. Such affordances highlight Spellburst's capacities for iteration and "accident-driven development" unlike platforms like Fork It.

4.4 Merging Nodes and Extracting Properties

As Ash iterates on her project, she realizes that she wants to combine visual aspects from two previous sketches along two different branches. However, manually merging code from different sketches can be time-consuming. Instead, Ash uses Spellburst's —merge→ feature to select Sketch 6 and Sketch 11 for merging (Figure 5 d). She does not provide any natural language input; moments later, Sketch 12 appears on screen. When Ash inspects the code for Sketch 12 in the code editor, she finds the following code comment, which is a natural language description of how the two sketches were merged: "Combine the planet system from [Code Snippet 1] with the mountain range background from [Code Snippet 2] which produces Sketch 12. The resulting sketch shows a planet system with a mountain range background that updates in real-time and is generated using Perlin noise."

In addition to merging sketches, Ash can use the prompt interface to extract specific sketch attributes as a new sketch (different from duplicating the entire sketch). For instance, Ash uses the —extraction→ operator to isolate the color gradient from Sketch 8. She's not sure where she plans to use it, but she wants to keep it readily accessible. In a different step in the exploration workflow, Ash can take a step back from the work as is and look at differences between the wide variety of sketches currently on the canvas. She takes the latest output from Sketch 11 and creates a new —diff→ operator to see a natural language summary of how Sketch 11 has changed from Sketch 10, as well as the differences between the sketches at a code level. She can then use this summary as input for future modifications.

Ash notices that Spellburst will occasionally generate code that does not run or code that conflicts with her expectations. Whenever this happens, Ash has two options: (1) step into the code editor to debug the code manually or (2) delete the node and regenerate the result. She does this various times throughout her session, pruning and growing her branches with ease. While the zoomable canvas gives her a "birds-eye view" similar to other node/canvas UIs, our structured auto-layout removes the need to "clean up" the canvas and makes it immediately clear how a sketch has progressed over time because it is a direct visualization of the version history tree. We also chose to focus on auto-layout so that artists would be focused on iteration rather than organizing their canvas, which was based on cognitive load concerns in our formative study.

Throughout her work with Spellburst, Ash benefits from the seamless integration of syntax and semantics, the intuitive Node View, and the powerful natural language interface. The system's

features enable her to focus on her artistic intentions and iterate on her generative art project with greater ease and control. By the end of this session, Ash has only begun her creative journey. She has successfully explored various corners of the design space, as sampled in Figure 3.

5 SPELLBURST

The core of Spellburst's implementation focuses on chaining together API calls to OpenAI [54] to generate p5.js sketches. This involves dynamically composing the prompt context based on manually written code, AI-generated code, previous prompts, and existing operator node data. Here we describe the prompt engineering approach for the key functionalities of Spellburst and provide details about our system implementation. The full set of prompts used in Spellburst are provided in the appendix (section A.2).

5.1 Prompt Design

To design the prompts, we iteratively explored a combination of prompting strategies based on a trial-and-error approach, using visual output as our guide. Unlike most LLM-powered coding assistants that mostly automate coding for one-off tasks, our prompts are intended to be "bridges" between p5.js source code and their visual outputs.

5.1.1 Code Generation. When first experimenting with prompting GPT-3.5 to output p5.js code, we used simple one-sentence questions (e.g., "Draw a p5.js code snippet that shows..."). However, we found that the model provided either code that did not work or output that did not align with our expectations. To address this shortcoming, we employed *few-shot prompting*. Few-shot prompting is derived from the technique of few-shot learning, where a model is trained on several related tasks in order to generalize well on new tasks with just a few examples [11]. This technique incorporates several examples of desired input and output examples within the prompt to steer the model's output in the right direction. Our examples consisted of both simple code snippets (e.g., a circle and a square on a screen) to more complex code snippets that allowed our model to decode abstract concepts (e.g., balls that bounce off the walls of the edges of the canvas). Below is a snippet of one of the examples we provided within our few-shot prompts:

Prompt: add a bunch more balls and make them bounce off the bounds

Output:

```
let numCircles = 20;
// Create an empty array to store the circles
let circles = [];
// Set up the canvas and create the circles
function setup() {
  createCanvas(700, 410);
  ...
}
```

After using few-shot prompting, we wanted to integrate the model's outputs within the Spellburst system. To create usable output, we forced the model to provide its answer using a specific *template* and *syntax*. At a high-level, templates are an effective prompting strategy, as they filter LLM output to eliminate choices that would have been unhelpful to the user [82]. For instance, our

prompts contained start and end tokens as well as extra instructions to make sure the code compiled and rendered correctly (e.g., limiting calls to the `draw()` function to ensure animation outputs render correctly). Getting LLMs to generate structured data consistently is an open research effort, with projects like `jsonformer` [66] or grammar-based sampling [22] allowing users to specify output schemas. However, we used the following restrictions in our prompt to guide the output:

Restrictions:

- Only respond with code in your output as a raw string.
- Be as efficient as possible with your implementations. When producing computationally intensive sketches, try to use optimization methods so they run more quickly.
- If you are ever asked to apply an animation, remember to always remove any calls of the `noLoop` function to make sure it actually animates.
- Comment your code with useful comments.
- Remember to be as efficient as possible with your implementations. When producing computationally intensive sketches, try to use optimization methods so they run more quickly.

Finally, we ran into specific problems that were unique to our use case of creative coding. While the use of natural language queries was helpful in making large jumps within the semantic space, the lack of the model’s explanation made it difficult to make more granular syntactic edits. To make the model output more explainable to users seeking finer control, we experimented with prompting the model to write code comments. We embedded these code comments within our few-shot prompts, outlining the purpose of every couple of lines. In this way, the code blocks served as chain-of-thought prompts, encouraging the model to think “step-by-step” to decompose hard tasks into smaller steps [10].

5.1.2 Prompt Auto-complete. Prompt authoring tools are currently underpowered and often involve simply typing into inputs without any form of auto-complete or facilitation for future direction. Unlike auto-complete for structured data like a programming language – where the system can easily match the input with a pre-existing set of options – natural language input can have multiple interpretations. Thus, it can be difficult to determine what the “correct” completion is, especially for creative text-generation tasks. Because of this ambiguity, we focused the Spellburst auto-complete on providing non-deterministic options to suggest interesting directions to take a prompt. Supporting exploration, rather than just “task execution” or search, is also important. By presenting a range of possible suggestions, auto-complete can encourage users to think more deeply about their search and discover new information that they may not have otherwise considered. The goal here is breaking past creative barriers and novelty.

With these design decisions in mind, we implemented prompt auto-complete by querying ChatGPT, again by using few-shot prompting to make the API simulate being an auto-complete engine. For context, we fed in the connected sketch to extract relevant keywords from (1) the current prompt being typed and (2) 5 examples of input-output prompt completion pairs.

5.1.3 Semantic Parameter Adjustment. Semantic parameters allow users to adjust sketches at the concept and prompt level, rather than needing to search for exactly which variables to manipulate in the source code. Our primary implementation method involves (1) extracting out key phrases from the input prompt, (2) generating a “semantic map” between these phrases and related global variable names, and (3) generating the desired code from the initial prompt and the semantic map with another API call. We found success by telling the LLM up-front to first name the global variables it would declare in relation to keywords in the prompt. This method mimics the data you might get from an attention “heatmap” between the input prompt and the output code, which is currently not possible to obtain due to the closed-source nature of OpenAI’s LLMs.

5.1.4 Merging Operations. Our implementation of merging is different from traditional merging algorithms. Existing methods, such as the three-way merge used in Git, try to combine code contents line-by-line without producing syntax errors. On the other hand, Spellburst’s merge functionality is LLM-powered, meaning that it tries to combine code contents in a semantically meaningful way. For example, it may take the physics behavior from a bouncing ball simulation and “merge” the color palette from an unrelated abstract art sketch. We call this process *semantic merging*.

To implement semantic merging, we used few-shot prompting to provide examples of expected outputs, a template and syntax to integrate the model’s response within Spellburst, and code comments to make the code explainable. One area we intend to explore more involves what a few-shot prompt example could look like. In some tests, we simply juxtaposed two `p5.js` sketches next to each other. For other tests, our merges were larger semantic jumps that would be hard to achieve by merely directly combining the two code snippets together. An example of one of our merging prompts can be found in A.2.

5.1.5 Developing a Taxonomy for Creative Coding Transformations. Finally, to align the natural language queries from users and the expected output from ChatGPT, we developed a taxonomy that classified different aspects of creative coding transformations. This taxonomy was developed by examining a variety of artistic taxonomies and adapting them to the context of creative coding. We organized the taxonomy into three main categories of transformation:

- (1) *Apply Transformation to Objects / Primitives / Marks:* This category deals with the individual elements or building blocks used in creative coding projects. These include properties such as color, shape, form, texture, thickness, waviness, curviness, and randomness*.
- (2) *Apply Transformation in Relation to the Plane/Canvas:* This category addresses how the elements in a creative coding project are transformed concerning the overall composition or canvas. Properties in this category include size, direction/orientation*, alignment*, white space*, movement, noisiness, symmetry*, scale/proportion*, hierarchy*, and randomness*.
- (3) *Apply Transformation to the Relationship Between Objects:* This category focuses on the relationships between elements in a creative coding project and how they can be transformed.

Properties in this category include nesting, direction/orientation*, repetition/pattern, proximity/spacing, alignment*, white space*, symmetry, contrast/emphasis, variety, balance, scale/proportion*, hierarchy*, unity, depths/layers, and randomness*.

Asterisks indicate properties that appear in two categories, illustrating the interconnectedness of these properties in creative coding transformations.

With this taxonomy, we searched and reviewed over 40 examples of creative coding work and labeled them according to this categorization. Subsequently, we wrote a code snippet corresponding to a change a participant could suggest in alignment with the taxonomy. Lastly, we crowd-sourced natural language queries that could be used to describe the transformation of the code from its original state to the state we coded. A curated number of crowd-sourced responses were then used as seed data to generate the few-shot examples for the ChatGPT API calls. Details on the crowd-sourced survey can be found in A.3.

5.2 Implementation Details

5.2.1 User Interface: The frontend of Spellburst is built using a combination of React [49], Reactflow [80], Jotai [34], and d3.js [8]. React is the foundation for our user interfaces, while Jotai allows us to store and manage the state of the graph's nodes and edges. For visualization, we used Reactflow to build the node-based interface for exploring variations through branching and merging, while d3.js is used to manage the hierarchical tree visualization and auto-layout of the graph.

Because the canvas involves constantly running multiple sketches at the same time – each of which with the ability to individually pause, play, and reset – we run each sketch in a separate `iframe`. To make the editing experience fast and responsive, we reload a sketch's `iframe` on every keystroke inside of the code editor. The most important quality of this automatic reloading is making sure the system is reactive and users can instantly respond to any visual feedback, which is important for discovering unexpected properties or behavior in sketches.

5.2.2 Backend and Language Model API: The Spellburst backend is built using Node.js [18] and Express.js [28] and uses OpenAI's ChatGPT [54] API for all language model functions, specifically the `gpt-3.5-turbo` model. The backend has several routes for modifying, merging, extracting, and auto-completing code, each dynamically generated based on the input.

5.2.3 Spellburst Graph: Each node and edge in the graph is represented as a JSON object. The properties of the respective JSON schemas help define the position, source code, and connections to other nodes on the graph. An example of the JSON objects for both components can be found in A.1. All operations on the graph are immutable by default, meaning that all modifications and adjustments to source code result in the creation of new nodes rather than overwriting the input. This immutable structure allows users to easily view the causal dependencies between operations and how sketches progress over time.

Whenever an operator node is re-run but is already connected to existing sketches, the connected sketches will automatically update

with the new code. Regenerated code only affects immediately connected sketches/edges one layer deep, so “out of date” code needs to be re-run one layer at a time. When a node is deleted from a graph, its descendants will remain in the graph. However, the descendants will be reattached to the deleted node's parent node. Although this method does not preserve the original causal structure of the graph, it facilitates visualization of the overall progression of how the initial sketch diverged over time.

6 EXPERT EVALUATION

In our evaluation, we aimed to determine whether artists are able to engage in exploratory creative coding using Spellburst. Concretely, we gathered feedback on Spellburst's support for (1) bridging semantic and syntactic spaces while creative coding through natural language prompting, (2) control over creative coding across large and small code increments, and (3) the usability and cognitive load while using Spellburst.

6.1 Method

We conducted a user study via Zoom with 11 expert creative coders. We recruited participants by contacting generative artists in our professional network and through social media messaging. One participant was a repeat participant from the formative study. Each session lasted approximately 75 minutes, and participants received \$30 for the time. Participants' demographic information can be found in the appendix (section A.6).

In each session, participants were given a walkthrough tutorial of Spellburst's key features and asked to complete a practice creative coding task. Once participants indicated familiarity with Spellburst, they proceeded to work on a set of creative coding tasks that required performing small changes to an initial starter code using prompts or direct code editing (e.g., tweaking the color of the circles in the sketch, four tasks total), executing large creative shifts using prompts (e.g., proposing an iteration that is a significant departure from the current code, four tasks total), and open-ended creative exploration using Spellburst's feature for 15 minutes. The juxtaposition of small and big changes maps to syntactic and semantic changes, respectively. By designing our two tasks like this, we wanted to formalize the system's ability to allow people to move between abstraction levels. Participants completed distinct tasks in each category using Spellburst and their typical IDE alongside ChatGPT [54]. We ran this as a within-subject comparison study, and the order of tool use was randomized. At the end of the study, each participant completed the NASA-TLX cognitive load questionnaire, a usability questionnaire, and finally participated in a 15-minute long open-ended discussion.

6.2 Findings

Using the snapshot of the canvas at the end of each user's run – encompassing the nodes, edges, and their types – we created a visualization of artists' exploration. The resulting visualization in Figure 6 shows distinct patterns of creative explorations for several participants in Spellburst, and appendix A.4 shows screenshots of Spellburst from two participants. Across tasks and open-ended exploration, participants engaged in rapid iteration, which ranged

from fine-grained editing to large semantic changes. Expansive patterns of divergent thinking were encouraged through Spellburst’s affordances with multiple creative pathways emerging from the root node. This can be especially seen for P6 in the upper left portion of Figure 6 which involved 10 different modifications from the root node, and P8, who merged after creating numerous variations. Spellburst can support different variations of depth and breadth in exploration, allowing users to easily explore a new area if they find that one line of iteration is no longer relevant.

Further, our analysis shows three types of creative exploration:

- (1) *Branching and Variations*: For some, the optimal way to use Spellburst was to create numerous branches off of the root node and then perform numerous modifications and merges from each branch. On the other hand, other participants found it beneficial to make either one or a few branches from the root node, perform a small number of modifications, and then create even more branches from those.
- (2) *Modify Nodes as Terminal Nodes*: While the majority of the terminal nodes are menu nodes – indicating sketches – a good number are also modifying nodes, with prompts to create variations that were ultimately unexplored.
- (3) *Merging after Several Modifications*: Merging often occurs after several modifications of a branch of changes. As a result, users often prefer to explore the possibilities within branches before merging two together, as opposed to merging together after smaller changes.

6.2.1 Prompt Based Editing. Across all sessions, participants responded positively to using prompts to initiate larger creative shifts. Although many expressed that it was their first time using LLM prompts in creative coding work, 73% agreed or strongly agreed that they liked using Spellburst’s interface and 90% agreed or strongly agreed that it was easy to learn to use the system (Figure 8). According to P7, when executing larger shifts, they don’t necessarily have a clear mapping to algorithms and attributes in the code. Instead, natural language is more expressive in communicating their creative intent: *“it matches to my expectations better on some of the conceptually large changes in the sense that I like didn’t have to be as precise...”* (P7) Similarly, P11 commented: *“I felt a better immediate response with the larger conceptual changes, just because you could at least get some of the feelings that I was trying to capture.”* In general, participants responded positively to the generated code outputs. As P6 commented about making a rainbow effect: *“This is exactly what I envisioned when I said, Make a rainbow. So that’s pretty cool. That’s close.”*

For smaller syntactic variations, participants had mixed opinions about using prompts. For example, P1 found the prompt-based interactions less useful when they already had a clear idea: *“I don’t really see this being good for when I have an idea of what I want. It seems way more frustrating to try to chat with it to give me what I want versus coding it myself.”* However, P9 commented about the prompt affordances that Spellburst allowed them to do things they would typically avoid: *“[with Spellburst] I found myself doing things that I wouldn’t normally do because for-loops are kind of a drag for me...but with Spellburst I was like, Oh, I don’t have to write out all that syntax for these for-loops.”* P2 commented that syntactic prompts could, in fact, make creative coding more

accessible to non-coders. These varied experiences suggest that the balance between semantic and syntactic prompting should be carefully considered in designing future systems like Spellburst.

When using prompts to generate code outputs, participants found the AI auto-complete suggestions beneficial in providing ideas. According to P10: *“I did find some inspiration in the suggestion...It took me down some paths that I didn’t really think I was gonna go down, and I always appreciate that like opportunity. so I can almost imagine like a little clippy-style AI...here’s a weird thing you could try with this output like just giving me ideas for where to go next.”* However, several participants expressed the need for interpretability of the generative outputs. P5 commented: *“I think the main thing I would want is more interpretability, like perhaps [currently] a little bit too much magic.”* and *“I find myself wanting to be able to turn each part [of the prompt] on and off to figure out which part is contributing to which.”*

The primary error encountered with Spellburst was related to bugs in the generated code that would either prevent that specific output from running or produce output that was far from the participant’s expectations. To address errors associated with AI generated code, we do allow artists to directly access the code editor. In cases where generated code did not run, the errors were isolated to one path, which participants typically addressed by trying again with a different prompt or going into the code editor. Several participants were eager to spend more time figuring out the generated errors in the code. However, the limited duration of the sessions served as a constraint. Future research could explore the trade-offs associated with using generated AI code that may be error-prone and require time and effort on manual bug-fixing modifications. P10 expressed how errors encountered through prompt generation could disrupt creative flow as it puts them in a “weird, intermediate space between image and code.”

6.2.2 Comparison of Spellburst with ChatGPT baseline. Based on feedback from using both Spellburst and their typical IDE alongside ChatGPT, a majority of artists found the visual interface, sketch organization, and branching features of Spellburst to be key factors that make it preferable when compared to ChatGPT. For example, P1 appreciated the branching capability of Spellburst, which enabled rapid creative exploration in comparison to a more linear process with ChatGPT: *“I think being able to go between the different branches was really interesting to me...modifying on a specific sketch that I see, like seeing the way that was laid out was really cool.”* Similarly, P9 commented how the visual iterations in Spellburst felt like a different process compared to ChatGPT and their regular IDE:

“...after the experience of using the raw ChatGPT version, I really did come to appreciate the sort of ease with which you could reason about the different versions, and very precisely sort of branch and duplicate and not have it be ambiguous sort of which one you were basic off of and sort of explore. The ChatGPT version felt very linear, whereas this one seemed to very much encourage branching exploration and making that sort of a first-class part of it. So that was cool.” (P9)

P8 even expressed how Spellburst could accelerate their creativity: *“I was going faster with the new interface for sure able to explore way more ideas per minute.”* However, in contrast, P4

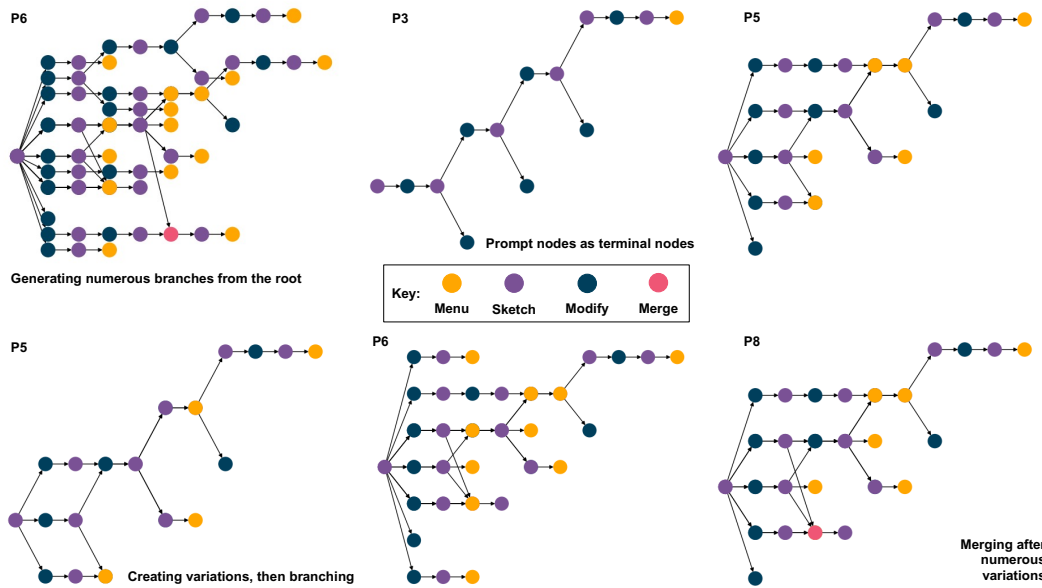


Figure 6: Abstract node visualizations of the canvas. The collated examples showcase the key affordances and behaviors associated with Spellburst.

mentioned reduced friction when using ChatGPT due to greater familiarity: “The ChatGPT interface is somewhat known to me. so I feel there is slightly less friction going in there [versus Spellburst].”

6.2.3 Usability of Spellburst’s Interface. Immediately following the evaluation study, we asked participants to complete both NASA Task Load Index (NASA-TLX) [25] and the Post-Study System Usability Questionnaire [42]. In the NASA-TLX survey results, participants found Spellburst to be the most mentally demanding ($mean = 9.9$, $SD = 4.8$). On the other hand, they found it to be the least physically demanding ($mean = 2.6$, $SD = 1.2$). There was notable variation in participants’ experiences in terms of success in accomplishing what they were asked to do with Spellburst ($mean = 11.4$, $SD = 5.2$), as well as mental demand ($mean = 9.9$, $SD = 4.8$) and stress with Spellburst ($mean = 7.1$, $SD = 4.4$).

In the PSSUQ survey results, we found the overall usability score for Spellburst as 7.67. In terms of sub-scales, the System Usefulness (SYSUSE) score was 8.26 which indicates that the system was relatively easy to use and learn. The Information Quality (INFOQUAL) score was 6.76, which suggests that while the information provided by the system was helpful, there is room for improvement in areas such as error messages, documentation, and information organization. The Interface Quality (INTERQUAL) score was 8.24, which indicates that participants found the system’s interface to be pleasant and visually appealing. Combined, these results indicate that Spellburst has a good level of usability, but some aspects, particularly information quality, could be further improved to enhance the user experience.

Participants appreciated Spellburst’s visual branching and merging without having to engage with Git-style commands. Many of the participants did not typically engage with Git for version control and appreciated how Spellburst could help seamlessly organize

their projects without too much overload, with P7 sharing how it can address their pitfall of focusing too much on organizing: “I find that anytime I get excited about trying to organize the creative project. All the excitement goes into the organizing.” Further, P10 appreciated the automatic branching functionality and the automatic rendering of the code, feeling that “this would work really well for me across my P5 sketches.” The visual version control mechanics of Spellburst also served to reduce creative friction for P8: “You don’t have to be scared that you’re going to lose a pretty state.”

Participants also discussed the potential for Spellburst to impact their workflow as P7 shares: “Maybe we do have something that’s maintained in a Git repository, and [Spellburst] is like a sandbox for me to change small pieces of code and see how it impacts the entire project.” To spur rapid iteration and creative exploration, P6 took advantage of running multiple prompts simultaneously and received an unanticipated benefit from the delay time associated with AI code generation: “It means you can start off ideas and not be influenced by the output of the first one when you’re thinking about the second one... this gives you time to continue thinking of ways to remix the previous ones without being influenced.”

However, some expressed a desire for bookmarking nodes or a way to clear unrelated content and focus on a specific branch without visual overload, as expressed by P8: “I want to just clear the stage and look at this one... otherwise it can be too much mind clutter.” The ability to zoom in and out at different levels of detail and give the user control over both output and focus space in the UI are considerations for future development.

6.2.4 Perceptions of impact on their creative practice. Participants expressed interest in a tool like this to support broader exploration and ideation in earlier stages of projects. Almost all found the tool beneficial for fast prototyping, with P1 likening the experience

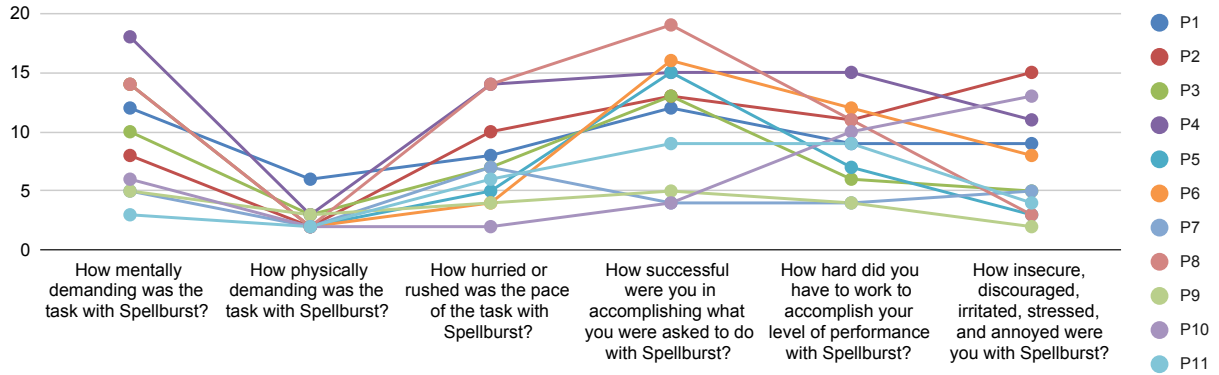


Figure 7: Participant Responses to the NASA-TLX Cognitive Load Questionnaire

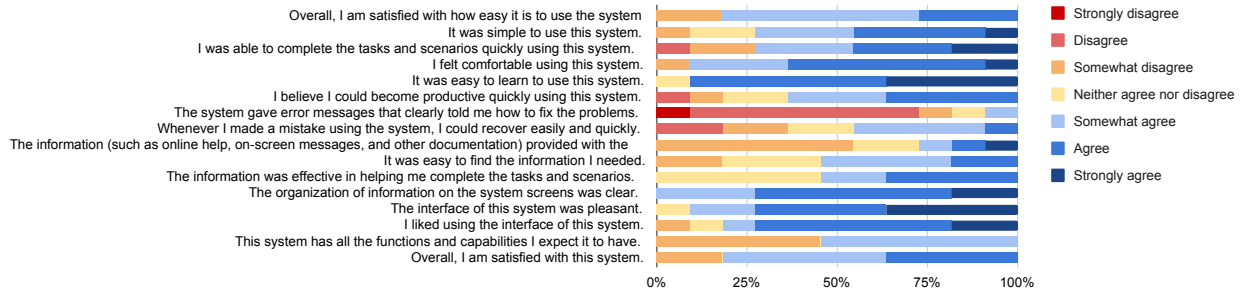


Figure 8: Participant Responses to Post-Study System Usability Questionnaire (PSSUQ)

“...as a practice like the same way you doodle on a napkin”. While several participants felt that the generated code typically did not map exactly to their expected output, it was perceived to help with ideation, with Spellburst uniquely designed to support visual navigation across different iterations. P9, who had limited prior experience using prompts to create code, noted how Spellburst encouraged her to “...reflect about the way I was thinking about code ... normally when I teach coding we’re giving very specific instructions to the computer. But now I’m giving less specific, more abstracted instructions ... and in this case they don’t need to be very specific [in order to be successful].”

However, some concerns were also raised about the responsible and ethical development and use of systems like Spellburst that leverage large language models with data trained from large amounts of information on the internet. This remains an open area of discussion, and the experts that we interviewed had conflicting views about artists’ rights to ownership, ranging from concern about plagiarizing to issues of ownership in art being historically muddy; for example, artists who lead teams of assistants may nominally take ownership of work that is actually much more collaborative. Specifically, some participants were troubled by the possibility of accidental plagiarism and a desire to give credit to prior artists, with P1 stating “I use other people’s code a lot, but it’s always credited,

and I try to be very clear about that in my sketches at the top [with comments acknowledging] that I’ve taken from others”. Perceptions of this did vary by individual, with P5 sharing that “I would be more concerned about copying somebody else’s visual style and less about copying the exact implementation of it.” P9 expressed concern about 1) bias in training sets “I’d be curious to see how it responds to race and gender” 2) potential lack of understanding amidst the layers of abstraction “...there’s less of a feeling that you have a grasp of what’s going on.”, and 3) skepticism toward AI tools developed by companies solely driven by profit “seeing some of them built on top of p5 feels like, Oh man, we were really trying to build a tool embedded around access and inclusion and they’re being wrapped by various layers. I would just question, what’s being prioritized in those tools?”

These perceptions tie back to prior research that shows how AI obscures key parts of the creative process that artists would have otherwise wanted to customize or understand [43] and that large language models can regurgitate copyrighted material without correct attribution [27], resulting in lawsuits about artists’ work being included as training data for LLMs without their consent [78]. Given the ethical and legal risks at hand, future work on Spellburst should require that LLMs provide proper citation of the source. This would also benefit users seeking more granularity and

explainability; they would be able to follow the original source and see more inspiration from real artists.

As some of our participants noted in the evaluation study, AI models carry inherent biases regarding race, gender, and other social constructs [5]. When artists prompt our system to interpret natural language phrases such as “make this sketch more feminine” or “make this sketch more aggressive,” social biases embedded in the model may be expressed in code or AI-generated descriptions of the code, as P9 demonstrated in her user study. As this field develops, we must attend to and contribute to the discussions around the responsible and ethical use of generative AI. Until then, we recognize and support the agency of each artist to decide whether or not to use AI-augmented CSTs such as Spellburst.

7 DISCUSSION

7.1 Utility of Spellburst

In addition to the workflow for expert creative coders described in Section 4, Spellburst can support a broader range of tasks for creativity and learning. We briefly describe a set of use cases to demonstrate the broader utility of Spellburst.

7.1.1 Learning Version Control for Creative Coding Novices. Version control management and creative coding involve techniques and tools that take time to learn. While this study focused primarily on expert usage, many participants commented on the playfulness of Spellburst as an exciting entry point for beginners. Spellburst could be adapted to help novices explore the full design space. This is especially important as novices lack the exposure to semantic constructs that they prefer (patterns, styles, forms) as well as syntactic constructs that they may employ (functions, parameters). As a result, they may get stuck more easily on either side of the divide. For example, a learner who is experimenting with trigonometric functions in p5.js for the first time may not know all possible effects of varying the parameter range. They will also not be able to map a commonly desired effect to the commonly used parameter range of $-\pi/4$ to $\pi/4$ radians. With Spellburst, a learner is free to cycle through many ideas without getting prematurely fixed in one corner of the design space or getting stuck while attempting to produce one desired effect. In addition, Spellburst provides learners with an intuitive interface for version control that allows them to save, reflect on, and debug their creative process.

7.1.2 Debugging. The structure of Spellburst as a node-based version control system also lends itself to supporting the practice of debugging for novices and experts alike. For example, imagine a user attempting to debug a complex p5.js sketch for a recursive fractal. If the current sketch is not working, a user can duplicate the existing sketch, inspect the code, and incrementally adjust the code until the bug is resolved. Then, they can merge the code back into the main branch. This can all be done with AI support via the natural language prompt interface.

7.1.3 Sharing and Documenting Work. We learned from our participants that creative coders often share and document their work for others, even when they work alone. Some of our study participants explained that they post screenshots of work-in-progress on social media networks to solicit feedback from peers. Others document

their process and code to be disseminated and “donated” to a public audience for remixing [70]. Imagine an artist who wants feedback on a new pendulum system they created using Spellburst. They post a link to their own Spellburst instance for everyone to see. A peer opens up the link and finds an intuitive and interactive account of the artist’s process. The peer sees the artist’s original intent in natural language with each prompt that was used; they also inspect the code that resulted at each step. The peer happens to prefer an earlier iteration rather than the latest sketch; because Spellburst preserves the non-linear nature of the work, the peer is able to easily discover it and point this out to the artist. A beginner who follows this artist on social media also receives the link. They are interested in how this artist created the pendulum; instead of scanning the workflow as the peer did, the beginner traces the artist’s process step-by-step to build their understanding of what the artist did.

7.2 Limitations and Future Work

While our evaluation study with generative artists provides valuable insights into Spellburst’s effectiveness, there are some limitations to consider, all of which open up avenues for future work.

Section 6.2 surfaced various limitations of our system related to unusable code, interface unfamiliarity, and cognitive load. Designing effective interfaces for handling generative AI errors remains an important need in future work. Unexpected results were especially common during merge operations. This is partly related to our design choice to favor ease of semantic merging over control: On one hand, the unexpected results led to playful or serendipitous discovery, especially during large creative jumps. On the other, our design limited users’ creative control when executing small, precise creative jumps. Future work could consider designs that strike a balance between exploration and control in merge operations, for example, by optionally allowing the user to explicitly specify and manipulate emphasis of aspects of interest from each input.

Further, while we made an attempt to demystify the output from various LLMs, our evaluation results indicate that it is hard to predict which prompts would be “effective” in generating desired variations. As a result, there is still no perfect mapping from semantic to syntactic changes, which could frustrate users who would otherwise get a lot from the platform. Initial future work could include further fine-tuning the model on examples using our taxonomy. Likewise, we could leverage other work done to make for better prompting experiences, including features to help add constraints to the generation process [30] as well as leveraging the strengths of the chat-based interface to find common ground [20] [65]. Future work should also investigate multi-modal prompting that can build referential connections between the visual canvas and text-based prompts.

Third, our study only explored a limited set of interactions with Spellburst. The 10 expert creative coders who participated in the evaluation are not representative of the richly diverse population of creative coders and artists. Future studies should involve larger samples with a wider range of backgrounds to better understand the system’s performance and utility. In addition, the tasks used in our evaluation did not cover all possible scenarios and use cases that creative coders might encounter. Exploring additional tasks,

particularly “medium-sized” jumps that are neither too drastic nor too minor, would provide a more comprehensive understanding of Spellburst’s affordances at various stages of the creative process (D6). Future work should investigate the effects of long-term, *in-situ* use on creative practice and the extent to which users can adapt to and benefit from the system over time and in the context of real-world artistic workflows. To support more nodes and edges, we may want to consider new approaches to scale the performance and usability of the interface.

Finally, large-scale creative work often requires an entire studio of artists and producers. For example, consider a game development team that is building a new underwater scene. Two game developers might need to explore many approaches for rendering the currents. They might start from the same sketch and create completely different interactive graphics. When they sit down to discuss, the developers realize that they appreciate the color scheme in one sketch but prefer the physic simulations of the second sketch. Future work should explore such collaborative use of Spellburst.

8 CONCLUSION

In this work, we present Spellburst, a creativity support tool designed to expand creative outputs for artists working with code. The node-based canvas integrates the act of creative exploration and tracking exploration history to reduce the burden on artists and enhance multiple creative strategies, including combinatorial and transformative creativity. Spellburst employs a large language model informed by crowdsourced data to support semantic and syntactic editing and exploration. Spellburst achieves a balance between larger creative shifts using prompts and fine-grained control through dynamic interfaces between prompt and code and direct code editing. Our evaluation with expert generative artists demonstrates that Spellburst enhances creative practice through rapid exploration while reducing the overhead of managing the exploration process.

ACKNOWLEDGMENTS

We are grateful to the reviewers and our study participants for their time and helpful feedback. We also thank Abdallah AbuHashem, Maxwell Bigman, Professor John Mitchell, Licia He, and Barron Webster for their input during the early iterations of this project.

REFERENCES

- [1] 2021. GitHub Copilot. <https://marketplace.visualstudio.com/items?itemName=GitHub.copilot>
- [2] Adobe. 2023. Adobe Firefly. <https://www.adobe.com/sensei/generative-ai/firefly.html>
- [3] ATLAS.ti. 2023. ATLAS.ti. <https://atlasti.com/atlas-ti-web>
- [4] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2022. Grounded Copilot: How Programmers Interact with Code-Generating Models. arXiv:2206.15000 [cs.HC]
- [5] Emily M. Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. 2021. On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?. In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency* (Virtual Event, Canada) (FAccT '21). Association for Computing Machinery, New York, NY, USA, 610–623. <https://doi.org/10.1145/3442188.3445922>
- [6] Mary Beth Kery and Brad A. Myers. 2017. Exploring exploratory programming. In *2017 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 25–29. <https://doi.org/10.1109/VLHCC.2017.8103446>
- [7] Margaret A Boden. 2010. *Creativity and art: Three roads to surprise*. Oxford University Press.
- [8] Mike Bostock. 2023. d3.js. <https://d3js.org/>
- [9] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative research in psychology* 3, 2 (2006), 77–101.
- [10] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 1877–1901. https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf
- [11] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models Are Few-Shot Learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) (NIPS'20). Curran Associates Inc., Red Hook, NY, USA, Article 159, 25 pages.
- [12] Cameron Burgess, Dan Lockton, Maayan Albert, and Daniel Cardoso Llach. 2020. Stumper: An Artboard-Oriented Creative Coding Environment. In *Extended Abstracts of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI EA '20). Association for Computing Machinery, New York, NY, USA, 1–9. <https://doi.org/10.1145/3334480.3382994>
- [13] Minsuk Chang, Stefania Druga, Alex Fiannaca, Pedro Vergani, Chinmay Kulkarni, Carrie Cai, and Michael Terry. 2023. The Prompt Artists. arXiv:2303.12253 [cs.HC]
- [14] Bangor School Department. 2012. G/T Student Questionnaire. <https://www.bangorschools.net/wp-content/uploads/2012/09/GT-Art-Screening-Questionnaire-Student1.pdf>
- [15] Derivative. 2023. Touch Designer. <https://derivative.ca/>
- [16] George Fiffeld. 2011. Drawing with Code: Works from the Anne and Michael Spalter Collection.
- [17] Charles Forceville. 2008. Metaphor in pictures and multimodal representations. *The Cambridge handbook of metaphor and thought* (2008), 462–482.
- [18] OpenJS Foundation. 2023. Node.js. <https://nodejs.org/en>
- [19] Jonas Frich, Lindsay MacDonald Vermeulen, Christian Remy, Michael Mose Biskjaer, and Peter Dalsgaard. 2019. Mapping the landscape of creativity support tools in HCI. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. 1–18.
- [20] Rinon Gal, Yuval Alaluf, Yuval Atzmon, Or Patashnik, Amit H. Bermano, Gal Chechik, and Daniel Cohen-Or. 2022. An Image is Worth One Word: Personalizing Text-to-Image Generation using Textual Inversion. arXiv:2208.01618 [cs.CV]
- [21] Berys Gaut. 2003. Creativity and imagination. *The creation of art* (2003), 148–173.
- [22] Georgi Gerganov. 2022. Llama : add grammar-based sampling. <https://github.com/ggerganov/llama.cpp/pull/1773>
- [23] Thomas R. G. Green and Marian Petre. 1996. Usability analysis of visual programming environments: a ‘cognitive dimensions’ framework. *Journal of Visual Languages & Computing* 7, 2 (1996), 131–174.
- [24] Stephen Halliwell. 2009. The aesthetics of mimesis. In *The Aesthetics of Mimesis*. Princeton University Press.
- [25] Sandra G Hart and Lowell E Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research. In *Advances in psychology*. Vol. 52. Elsevier, 139–183.
- [26] Björn Hartmann, Loren Yu, Abel Allison, Yeonsoo Yang, and Scott R Klemmer. 2008. Design as exploration: creating interface alternatives through parallel authoring and runtime tuning. In *Proceedings of the 21st annual ACM symposium on User interface software and technology*. 91–100.
- [27] Peter Henderson, Xuechen Li, Dan Jurafsky, Tatsunori Hashimoto, Mark A. Lemley, and Percy Liang. 2023. Foundation Models and Fair Use. arXiv:2303.15715 [cs.CY]
- [28] TJ Holowaychuk. 2023. Express.js. <https://expressjs.com/>
- [29] Lingdong Huang. 2022. Srcsnap: Screenshot-driven version tracking. <https://github.com/LingDong-/srcsnap>
- [30] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A. Efros. 2018. Image-to-Image Translation with Conditional Adversarial Networks. arXiv:1611.07004 [cs.CV]
- [31] Jennifer Jacobs, Joel Brandt, Radomir Mech, and Mitchel Resnick. 2018. Extending manual drawing practices with artist-centric programming tools. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. 1–13.
- [32] Jennifer Jacobs, Sumit Gogia, Radomir Měch, and Joel R Brandt. 2017. Supporting expressive procedural art creation through direct manipulation. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 6330–6341.

- [33] Vanita Jain, Piyush Agrawal, Subham Banga, Rishabh Kapoor, and Shashwat Gulyani. 2019. Sketch2Code: transformation of sketches to UI in real-time using deep neural network. *arXiv preprint arXiv:1910.08930* (2019).
- [34] Daishi Kato. 2023. Jotai. <https://jotai.org/>
- [35] Mary Beth Kery. 2021. Designing Effective History Support for Exploratory Programming Data Work. (2021).
- [36] Mary Beth Kery, Amber Horvath, and Brad A Myers. 2017. Variolite: Supporting Exploratory Programming by Data Scientists. In *CHI*, Vol. 10. 3025453–3025626.
- [37] Kevin Gonyop Kim, Richard Lee Davis, Alessia Eletta Coppi, Alberto Cattaneo, and Pierre Dillenbourg. 2022. Mixplorer: Scaffolding Design Space Exploration through Genetic Recombination of Multiple Peoples' Designs to Support Novices' Creativity. In *CHI Conference on Human Factors in Computing Systems*. 1–13.
- [38] Kevin Gonyop Kim, Richard Lee Davis, Alessia Eletta Coppi, Alberto Cattaneo, and Pierre Dillenbourg. 2022. Mixplorer: Scaffolding Design Space Exploration through Genetic Recombination of Multiple Peoples' Designs to Support Novices' Creativity. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems* (New Orleans, LA, USA) (*CHI '22*). Association for Computing Machinery, New York, NY, USA, Article 308, 13 pages. <https://doi.org/10.1145/3491102.3501854>
- [39] Oded M Kleinmuntz, Tal Ivancovsky, and Simone G Shamay-Tsoory. 2019. The two-fold model of creativity: the neural underpinnings of the generation and evaluation of creative ideas. *Current Opinion in Behavioral Sciences* 27 (2019), 131–138.
- [40] Aaron D Knochel and Ryan M Patton. 2015. If art education then critical digital making: Computational thinking and creative code. *Studies in Art Education* 57, 1 (2015), 21–38.
- [41] Chinmay Kulkarni, Stefania Druga, Minsuk Chang, Alex Fiannaca, Carrie Cai, and Michael Terry. 2023. A Word is Worth a Thousand Pictures: Prompts as AI Design Material. *arXiv:2303.12647* [cs.HC]
- [42] James R Lewis. 1992. Psychometric evaluation of the post-study system usability questionnaire: The PSSUQ. In *Proceedings of the human factors society annual meeting*, Vol. 36. Sage Publications Sage CA: Los Angeles, CA, 1259–1260.
- [43] Jingyi Li, Sonia Hashim, and Jennifer Jacobs. 2021. What We Can Learn From Visual Artists About Software Development. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. 1–14.
- [44] Zach Lieberman, Theodore Watson, Arturo Castro, et al. 2009. Openframeworks. URL <http://openframeworks.cc/about> (2009).
- [45] Vivian Liu and Lydia B Chilton. 2022. Design Guidelines for Prompt Engineering Text-to-Image Generative Models. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems* (New Orleans, LA, USA) (*CHI '22*). Association for Computing Machinery, New York, NY, USA, Article 384, 23 pages. <https://doi.org/10.1145/3491102.3501825>
- [46] John Maloney, Mitchel Resnick, Natalie Rusk, Brian Silverman, and Evelyn Eastmond. 2010. The scratch programming language and environment. *ACM Transactions on Computing Education (TOCE)* 10, 4 (2010), 1–15.
- [47] Lauren McCarthy. 2023. p5.js. <https://p5js.org/>
- [48] Lauren McCarthy, Casey Reas, and Ben Fry. 2015. *Getting started with P5.js: Making interactive graphics in JavaScript and processing*. Maker Media, Inc.
- [49] Meta. 2023. React. <https://react.dev/>
- [50] Hiroaki Mikami, Daisuke Sakamoto, and Takeo Igarashi. 2017. Micro-Versioning Tool to Support Experimentation in Exploratory Programming. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems* (Denver, Colorado, USA) (*CHI '17*). Association for Computing Machinery, New York, NY, USA, 6208–6219. <https://doi.org/10.1145/3025453.3025597>
- [51] Hussein Mozannar, Gagan Bansal, Adam Fournery, and Eric Horvitz. 2022. Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming. *arXiv:2210.14306* [cs.SE]
- [52] Brad A Myers. 1990. Taxonomies of visual programming and program visualization. *Journal of Visual Languages & Computing* 1, 1 (1990), 97–123.
- [53] Bernard A Nijstad, Wolfgang Stroebe, and Hein FM Lodewijckx. 2002. Cognitive stimulation and interference in groups: Exposure effects in an idea generation task. *Journal of experimental social psychology* 38, 6 (2002), 535–544.
- [54] OpenAI. 2023. ChatGPT. <https://chat.openai.com/>
- [55] OpenAI. 2023. Dall-E 2. <https://openai.com/product/dall-e-2>
- [56] Fatih Kursat Ozenc, Miso Kim, John Zimmerman, Stephen Oney, and Brad Myers. 2010. How to Support Designers in Getting Hold of the Immaterial Material of Software. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Atlanta, Georgia, USA) (*CHI '10*). Association for Computing Machinery, New York, NY, USA, 2513–2522. <https://doi.org/10.1145/1753326.1753707>
- [57] Stefan Palan and Christian Schitter. 2018. Prolific.ac—A subject pool for online experiments. *Journal of Behavioral and Experimental Finance* 17 (2018), 22–27. <https://doi.org/10.1016/j.jbef.2017.12.004>
- [58] Kylie Peppeler and Yasmin Kafai. 2005. Creative coding: Programming for personal expression. *Retrieved August 30, 2008* (2005), 314.
- [59] Anna L. Phillips. 2011. The Algorists. *American Scientist* 99, 2 (Mar 2011), 126. <https://www.proquest.com/scholarly-journals/algorists/docview/878896671/se-2> Copyright - Copyright Sigma XI-The Scientific Research Society Mar/Apr 2011; Document feature - Photographs; Last updated - 2017-11-18; CODEN - AMSCAC.
- [60] The Creators Project. 2012. How to Draw With Code | Case Reas.
- [61] Eric Rawn, Jingyi Li, Eric Paulos, and Sarah Chasins. 2023. Understanding Version Control as Material Interaction with Quickpose. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*.
- [62] Casey Reas, Ben Fry, and John Maeda. 2007. *Processing: A Programming Handbook for Visual Designers and Artists*. The MIT Press.
- [63] Replit. 2022. Ghostwriter - code faster with AI. <https://replit.com/site/ghostwriter>
- [64] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2021. High-Resolution Image Synthesis with Latent Diffusion Models. *arXiv:2112.10752* [cs.CV]
- [65] Nataniel Ruiz, Yuanzhen Li, Varun Jampani, Yael Pritch, Michael Rubinstein, and Kfir Aberman. 2022. DreamBooth: Fine Tuning Text-to-image Diffusion Models for Subject-Driven Generation. (2022).
- [66] Rahul Sengottuvelu. 2023. Jsonformer: A Bulletproof Way to Generate Structured JSON from Language Models. <https://github.com/1rgs/jsonformer>
- [67] Paul Shen. 2023. natto.dev. <https://natto.dev/>
- [68] Daniel Shiffman, Shannon Fry, and Zannah Marsh. 2012. *The nature of code*. D. Shiffman.
- [69] Evan Shimizu, Matthew Fisher, Sylvain Paris, James McCann, and Kayvon Fatahalian. 2020. Design Adjectives: A Framework for Interactive Model-Guided Exploration of Parameterized Design Spaces. In *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology*. 261–278.
- [70] Ben Shneiderman. 2002. Creativity support tools. *Commun. ACM* 45, 10 (2002), 116–120.
- [71] Ben Shneiderman. 2007. Creativity support tools: accelerating discovery and innovation. *Commun. ACM* 50, 12 (2007), 20–32.
- [72] Sarah Sterman, Molly Jane Nicholas, and Eric Paulos. 2022. Towards Creative Version Control. In *CM Conference on Computer-Supported Cooperative Work and Social Computing*.
- [73] Anselm Strauss and Juliet Corbin. 1990. *Basics of qualitative research*. Sage publications.
- [74] Kesler Tanner, Naomi Johnson, and James A. Landay. 2019. Poirot: A Web Inspector for Designers. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland Uk) (*CHI '19*). Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/3290605.3300758>
- [75] Michael Tempel. 2017. Generative art for all. *Journal of Innovation and Entrepreneurship* 6, 1 (2017), 1–14.
- [76] Mark Turner. 2006. *The artful mind: Cognitive science and the riddle of human creativity*. Oxford University Press.
- [77] Bret Victor. 2012. Learnable Programming : designing a programming system for understanding programs. (2012). <http://worrydream.com/LearnableProgramming/>
- [78] James Vincent. 2023. AI art tools Stable Diffusion and Midjourney targeted with copyright lawsuit. *The Verge* (Jan 2023). <https://www.theverge.com/2023/1/16/23557098/generative-ai-art-copyright-legal-lawsuit-stable-diffusion-midjourney-deviantart>
- [79] Adrian Ward and Geoff Cox. 1999. How I Drew One of My Pictures: or, The Authorship of Generative Art. In *International Conference on Generative Art*. Generative Design Lab Milan.
- [80] webkid. 2023. Reactflow. <https://reactflow.dev/>
- [81] Nathaniel Weinman, Steven M. Drucker, Titus Barik, and Robert DeLine. 2021. Fork It: Supporting Stateful Alternatives in Computational Notebooks. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems* (Yokohama, Japan) (*CHI '21*). Association for Computing Machinery, New York, NY, USA, Article 307, 12 pages. <https://doi.org/10.1145/3411764.3445527>
- [82] Jules White, Quichen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C. Schmidt. 2023. A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. *arXiv:2302.11382* [cs.SE]
- [83] Loutfouz Zaman, Wolfgang Stuerzlinger, Christian Neugebauer, Rob Woodbury, Maher Elkhaldi, Naghmi Shireen, and Michael Terry. 2015. Gem-ni: A system for creating and managing alternatives in generative design. In *Proceedings of the 33rd annual ACM conference on human factors in computing systems*. 1201–1210.

A APPENDIX

A.1 JSON Schemas for Graph Data

Below is an example of the JSON schema for a node.

```
{
  "width": 300,
  "height": 324,
  "id": "wgtt0s",
  "type": "sketch",
  "data": {
    "sourceNode": "root",
    "sourceCode": "\nfunction setup() {\n  createCanvas(400, 400);\n  background(255);\n  strokeWeight(2);\n  stroke(0);\n}\n\nfunction draw() {\n\n}
    ",
    "size": {
      "width": 300,
      "height": 300
    }
  },
  "position": {
    "x": 0,
    "y": 0
  },
  "sourcePosition": "right",
  "targetPosition": "left",
  "selected": false,
  "positionAbsolute": {
    "x": 0,
    "y": 0
  }
}
```

Below is an example of a JSON object for an edge:

```
{
  "id": "wgtt0s=>ic45uc",
  "source": "wgtt0s",
  "target": "ic45uc",
  "type": "connected",
  "selected": false
}
```

A.2 Prompts for LLMs

Below are the prompts we fed into the ChatGPT API to perform each of Spellburst's core operators. The ChatGPT API takes in an array of messages in the request body following this order: [System Configuration, Dynamic Context, User Input]. The System Configuration is a preset preamble for each route based on the task at hand (modification, merging, etc.). The Dynamic Context is a series of user and assistant messages to demonstrate examples of the kinds of output each route should output, where the assistant is primarily tasked with outputting code (rather than responding with natural language chat). The last user example that's sent is formatted in the same way as the previous, generated user messages in the dynamic context, and the final response that the user sees as the sketch is generated is a final assistant output.

With these details in mind, we provide the System Configuration and Dynamic Context for reproducibility.

A.2.1 Restrictions. For each prompt, we append a set of restrictions. These restrictions helped structure ChatGPT's return values and also addressed common issues that occurred during experimentation. The string of these restrictions is defined as @BASE_RESTRICTIONS, and is defined in future prompts.

Restrictions:

- Only respond with code in your output as a raw string.
- Be as efficient as possible with your implementations. When producing computationally intensive sketches, try to use optimization methods so they run more quickly.
- If you are ever asked to apply an animation, remember to always remove any calls of the noLoop function to make sure it actually animates.
- Comment your code with useful comments.
- Remember to be as efficient as possible with your implementations. When producing computationally intensive sketches, try to use optimization methods so they run more quickly.

A.2.2 Modification. The System Configuration prompt is:

You are a creative coding software engineer focused on creating visually stunning graphics, physics simulations, biological simulations, and data visualizations using p5.js. You are excellent at a few things: creating p5.js sketches, modifying p5.js sketches with natural language prompts, and blending multiple sketches together by merging their code in semantically meaningful ways.

@BASE_RESTRICTIONS

The Dynamic Context is:

```
const VARIATION_CONTEXT = [
  {
    "role": "user",
    "content": JSON.stringify({
      code: `let x = 100;
let y = 100;
function setup() {
  createCanvas(700, 410);
};
function draw() {
  background(0);
  fill(255);
  rect(x, y, 50, 50);
};`,
      variationPrompt: "add a bunch more balls and make them bounce off the bounds"
    })
  },
  {
    "role": "assistant",
    "content": `//${START_TOKEN}
let numCircles = 20;
// Create an empty array to store the circles
let circles = [];
// Set up the canvas and create the circles
function setup() {
  createCanvas(700, 410);
  for (let i = 0; i < numCircles; i++) {
    circles.push({
      // randomly set the x and y coordinates of each circle within the canvas
      x: Math.floor(Math.random() * 700),
      y: Math.floor(Math.random() * 410),
      // set the radius of each circle
      radius: 10,
      // set the x and y velocity of each circle to a random value between 0 and 0.5
      xVel: Math.random() * 0.5,
      yVel: Math.random() * 0.5,
    });
  }
};`
  }
];
```

```

    }
};
function draw() {
    background(0);
    // loop through each circle in the array and move it
    // according to its velocity
    for (let i = 0; i < circles.length; i++) {
        let cir = circles[i];
        cir.x += cir.xVel;
        cir.y += cir.yVel;

        // if a circle reaches the edge of the canvas,
        // reverse its direction
        if (cir.x >= width || cir.x <= 0) {
            cir.xVel *= -1;
        }
        if (cir.y >= height || cir.y <= 0) {
            cir.yVel *= -1;
        }

        // set the fill color to white and draw the circle at
        // its current position
        fill(255);
        ellipse(cir.x, cir.y, cir.radius);
    }
};
//${END_TOKEN}`
}
]

```

A.2.3 Merging. The System Configuration prompt is:

Given two p5.js code snippets, generate a new code snippet that combines the functionality of both snippets. The output code snippet should be valid p5.js code and should have as much similarity as possible to the original inputs.

First, you'll begin your generation by creating a "merge prompt". This can either be supplied by the user, otherwise you will create it.

It should follow this format: "Combine [Feature A] from [Code Snippet 1] with [Feature B] from [Code Snippet 2]. The resulting code should [Describe desired functionality]."

In this format, you would fill in the placeholders with the relevant information for your specific merge prompt. For example: `/*Combine the animation loop from Code Snippet 1 with the mouse-interactivity of Code Snippet 2. The resulting code should draw a looped animation that responds to user mouse movement by changing its direction and speed in real-time.*/`

Then you'll produce the relevant p5 code according to the prompt and the format provided in the following examples.

@BASE_RESTRICTIONS

- Remember to include the merge prompt inside of code comments.

The Dynamic Context is:

```
const exampleInput = {
```

```

    firstCode: `let angle = 0;
    let r = 100;
    function setup() {
        createCanvas(400, 400);
        background(220);
    }
    function draw() {
        translate(width / 2, height / 2);
        rotate(angle);
        strokeWeight(2);
        stroke(0);
        line(0, 0, r, 0);
        angle += 0.05;
    }
    `,
    secondCode: `let x, y;
    let speed = 3;
    function setup() {
        createCanvas(400, 400);
        x = width / 2;
        y = height / 2;
    }
    function draw() {
        background(220);
        ellipse(x, y, 50, 50);
        x += speed;
        if (x > width || x < 0) {
            speed *= -1;
        }
    }
    `
}

const exampleOutput = `//${START_TOKEN}
/* Combine the rotating line animation from Snippet 1
with the bouncing ball behavior from Snippet 2. The
resulting code should draw a rotating line that
bounces off the walls of the canvas and leaves a
trail of dots or other shapes */

let angle = 0;
let r = 100;
let x, y;
let speed = 3;
function setup() {
    createCanvas(400, 400);
    x = width / 2;
    y = height / 2;
    background(220);
}
function draw() {
    translate(width / 2, height / 2);
    rotate(angle);
    strokeWeight(2);
    stroke(0);
    line(r, 0, x - width / 2, y - height / 2);
    angle += 0.05;
    ellipse(x, y, 5, 5);
    x += speed;
    if (x > width || x < 0) {
        speed *= -1;
    }
    y = height / 2 + sin(x * 0.02) * 100;
}
//${END_TOKEN}
`

const MERGE_CONTEXT = [
    {
        role: "user",
        content: JSON.stringify(exampleInput)
    },
    {
        role: "assistant",
        content: exampleOutput
    }
]

```

A.2.4 Autocomplete. The System Configuration prompt is: Your role is to provide autocomplete results for a natural language prompt-editor for creating p5.js sketches.

If the input prompt is an incomplete sentence, provide results that continue the sentence. If the input prompt is a complete sentence, provide more complete sentences.

You will *always* provide results or suggestions, even if the input seems incomplete.

Provide maximum 3 (three) suggestion results. Do not respond with any english. You are not a chat. You are simply returning arrays of data.

The Dynamic Context is:

```
const AUTOCOMPLETE_CONTEXT = [
  {
    "role": "user",
    "content": "make an intricate tree with branches that twist and turn, gradually tapering off into smaller and smaller branches."
  },
  {
    "role": "assistant",
    "content": "[\"add variation in color and thickness to branches\", \"randomize branch angles and lengths\", \"incorporate falling leaves or flowers\"]"
  },
  {
    "role": "user",
    "content": "make it more"
  },
  {
    "role": "assistant",
    "content": "[\"colorful\", \"sporadic and physical\", \"like a surreal drawing\"]"
  },
  {
    "role": "user",
    "content": "draw numerous small particles"
  },
  {
    "role": "assistant",
    "content": "[\"that are attracted to each other with a gravity well\", \"that respond to user input to change particle behavior\", \"that collide with each other\"]"
  },
  {
    "role": "user",
    "content": "create an abstract and "
  },
  {
    "role": "assistant",
    "content": "[\"visually striking piece of art using perlin noise\", \"experiment with color gradients and blending modes\", \"incorporate user input for dynamic patterns\"]"
  },
]
```

A.2.5 Extraction. The System Configuration prompt is: You are the most experienced creative coding assistant in the world who is focused on creating visually stunning graphics, physics simulations, biological simulations, and data visualizations using p5.js. You can help answer

coding questions, write code, and change code. Specifically, you are excellent at answering questions about p5.js sketches.

You have read countless articles about building interactive art and graphics, and have read everything from the p5.js API documentation (<https://p5js.org/reference/>), as well as all of the "Nature of Code" articles and tutorials (<https://natureofcode.com/book/>).

A.2.6 Diffing. The System Configuration prompt is: You are the most experienced creative coding assistant in the world who is focused on creating visually stunning graphics, physics simulations, biological simulations, and data visualizations using p5.js.

You have read countless articles about building interactive art and graphics, and have read everything from the p5.js API documentation (<https://p5js.org/reference/>), as well as all of the "Nature of Code" articles and tutorials (<https://natureofcode.com/book/>).

Compare the two pieces of p5.js code. In no more than 5 sentences, describe how they are similar and different. Focus on the content of each sketch, their properties (such as color and stroke), and code-level differences. Don't propose a function.

A.3 Crowd-sourcing Image Transformations

A.3.1 Design of Survey. After developing our taxonomy, coding the examples, and selecting the top 20 representative examples, we recruited participants from the Prolific platform [57], targeting adult (18+), English first language speakers with sufficient art experience from the USA and with access to a desktop computer. The initial set of age, location, and language constraints narrowed the eligible pool of participants from roughly 120,000 to 37,000. With this participant pool in mind, we implemented a pre-screening questionnaire in Prolific to assess the participants' art experience and self-perceptions in relation to art. The questions were inspired from a gifted and talented art screening questionnaire [14]. A total of 370 participants completed the initial pre-screen survey.

We then invited the eligible participants to complete a series of crowdsourced tasks, each of which involved them providing a natural language prompt to transform an image from one state to the next. The tasks were designed to only elicit participants' natural language descriptions, without exposing them to the underlying code.

Each task included an initial starter image output of a creative coding project and a final image (see Figure 9 for an example). Participants were asked to provide a prompt that they believed would sufficiently describe the transformation from the starter image to the final image. Incomplete and timed out crowdsourced tasks were flagged by Prolific and discarded. In total, we collected 12 prompts for each of the 20 images with input from 52 unique participants on at least one prompt.



Imagine you are asking an artist to take image 1 and make something in the style of image 2 using a digital tool. You only have one shot and need to be as descriptive as possible. Keep in mind that the artist cannot see image 2; they have to adjust the image purely based on the text you provide below.

How would you describe to the artist what changes to make to image 1? Write as if you are directly describing the changes.

Figure 9: Example crowdsourced task

We categorized the crowd-sourced responses into three broad categories: semantic descriptions (“make it look like an iceberg melting”), syntactic descriptions (“increase the stroke weight of the lines”), and *combined semantic and syntactic descriptions* (“randomly vary the thickness like tree branches”). We found that a common pattern for successful *combined semantic and syntactic descriptions* involved reference to a taxonomy term as well as reference to a simile or metaphor.

A.3.2 Example Prompts. Here are some example prompts from the survey:

- *With the circle image, create a finale image that has many circles echoing and rippling out all around it. As the shapes ripples out, the shades fade out in greyscale until they're almost invisible. They distorted and change shape and are no longer perfect circles. To achieve this, lower the images levels by 30%. Then multiple the original circle layer and lower the levels 5% more on the new layer. With the new circle layer than distort and manipulate it's shape. Repeat these steps again, and again, until dozens of distorted shapes are echoing out from the original circle layer.*
- *Take the different sides of the line of the provided image (circle) and use the tool to stretch the line in and out making a sort of wave effect around the whole circle. Then copy this line over itself many times at different points either farther inside the original circle line or farther outside the original circle line.*
- *instead of a circle, it would look like multiple pencil lines overlapping to create an organic looking shape.*
- *You will need to add a bunch of lines to image 1. You will need to draw an irregular circle multiple times.*
- *I would open it up so that it is no longer a circle and make it look more vibrant and less fluid. It will have a distressed quality and will look a little frayed. It will have blurred lines that will overlap one another. It won't be a closed circle and will be a lot larger. The blackness will also be a little more like a grey color.*

A.4 Example Spellburst Evaluation Output

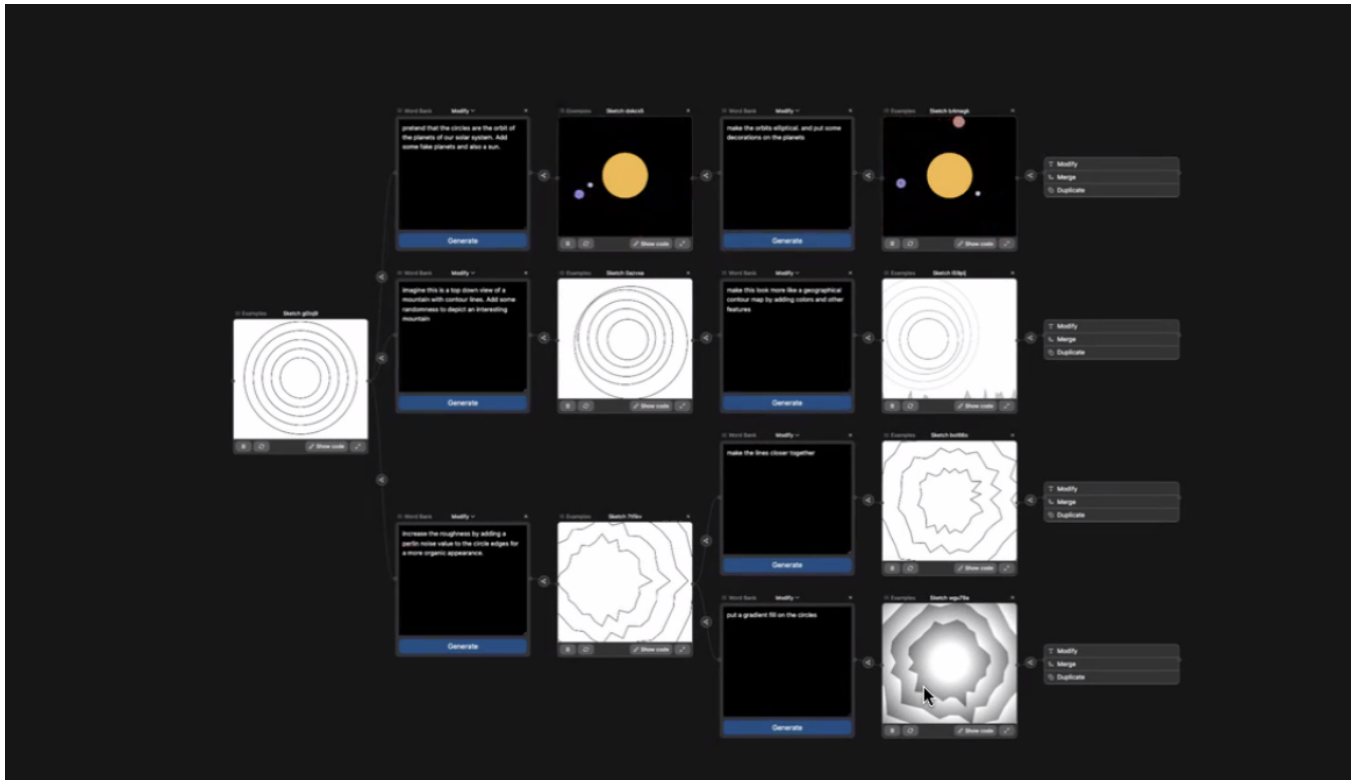


Figure 10: Evaluation study sketch from P6 indicating encompassing several variations on concentric circles starter code.

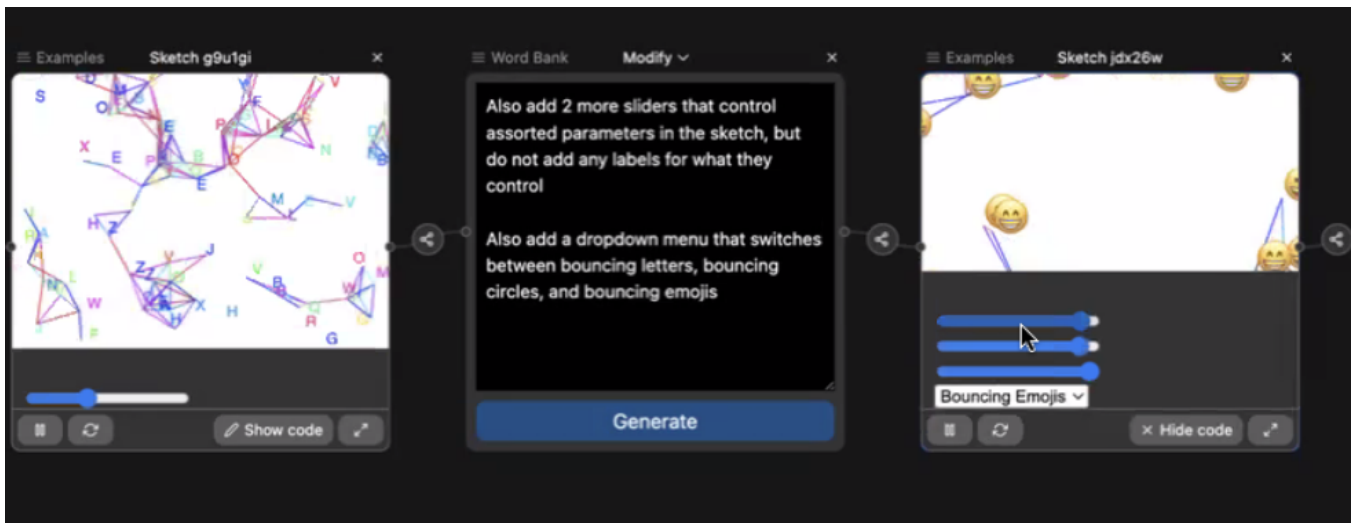


Figure 11: Evaluation study sketch from P8 indicating the user's generation of a provisional UI in the form of sliders to support their creative exploration.

A.5 System Architecture

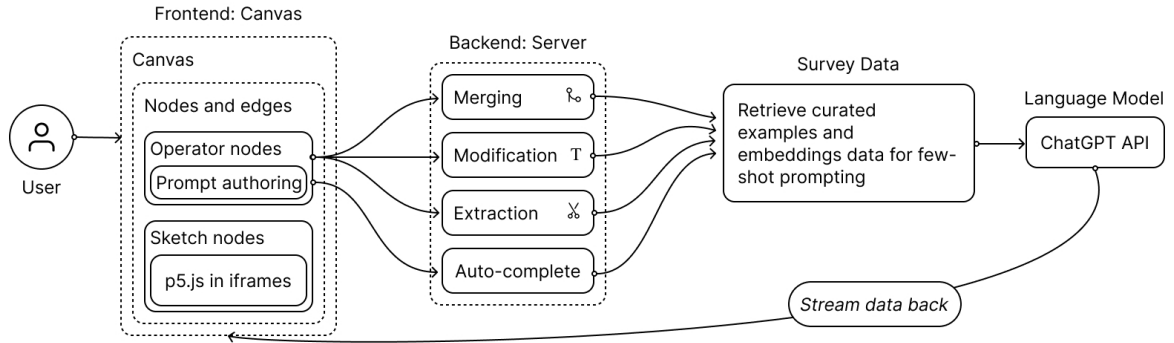


Figure 12: System architecture of Spellburst, showing the connection between the front-end Canvas and the server that calls the ChatGPT API.

A.6 Demographic Information of Participants

	Gender	Experience (Years)	Background	Tools
U1	M	5+	Creative Coding Educator	p5.js, Python
U2	F	5+	Generative Artist	Javascript, Python
U3	F	5+	Generative Artist	Javascript p5.js
U4	M	3-5	Visual Designer	Adobe, Figma React.js
U5	M	5+	Software Engineer	Javascript
U6	M	5+	Software Engineer	Javascript, HTML, CSS
U7	M	5+	Game Developer	Unreal Engine
U8	F	5+	Visual Designer	Figma, SVG Art
U9	M	5+	Generative Artist	Javascript
U10	M	5+	Generative Artist	Javascript

Table 1: Generative Study Participants Demographics

	Gender	Experience (Years)	Background	Tools
P1	F	2.5	Generative Artist	C++, Javascript
P2	M	10-11	UI Designer, Creative Coder	HTML, CSS Javascript
P3	M	10-15	Software Engineer	p5.js, 3D Modeling
P4	M	4	HCI Researcher	Sketch-n-Sketch
P5	M	6	Creative Coder	LED and Video Art
P6	M	5-6	Generative Artist, Software Engineer	HTML, CSS, p5.js
P7	M	8-10	Digital Artist, Data Scientist	Python, Blender
P8	M	5	Creative Coder, College Student	Processing, Python
P9	F	15+	Creative Coder, Open Source Creator	Swift, Java, HTML, CSS
P10	M	10	Visual Designer	HTML, CSS, 3D Modeling
P11	F	4	Creative Coder	p5.js, Unity (C#)

Table 2: Expert Evaluation Participants Demographics